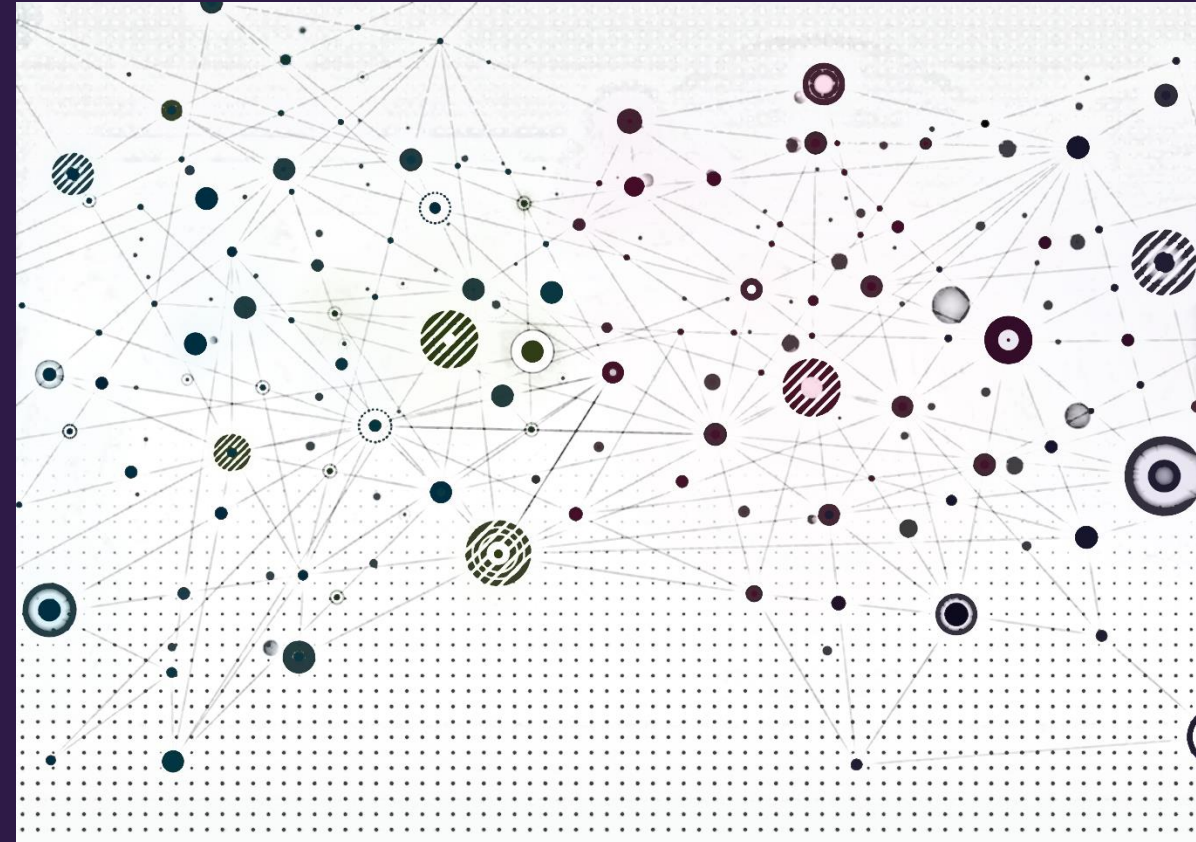


Reasoning on knowledge graphs





Overview

- Queries on knowledge graphs
- Predictive queries
- Predictive path queries with TransE
- Predictive conjunctive queries with Query2Box



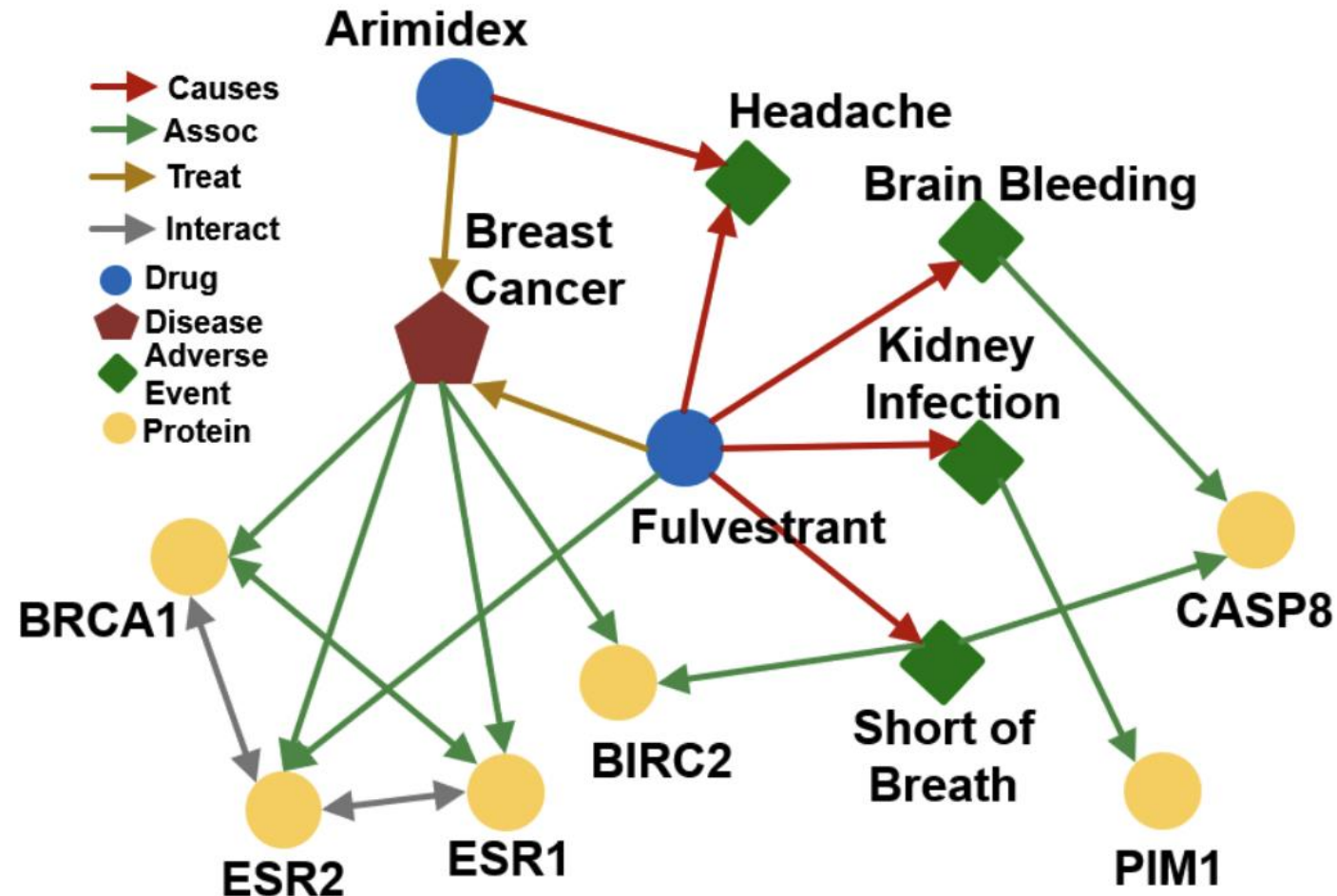
Queries on knowledge graphs





Queries (reasoning) on knowledge graphs

- How do we perform multi-hop (edge) reasoning over knowledge graphs (KGs)?





Types of multi-hop queries

Can we answer complex queries on a KG?

Query Types	Examples: Natural Language Question, Query
One-hop Queries	What adverse event is caused by Fulvestrant? (e:Fulvestrant, (r:Causes))
Path Queries	What protein is associated with the adverse event caused by Fulvestrant? (e:Fulvestrant, (r:Causes, r:Assoc))
Conjunctive Queries	What is the drug that treats breast cancer and caused headache? ((e:BreastCancer, (r:TreatedBy)), (e:Migraine, (r:CausedBy)))

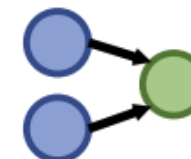
In this lecture, we only focus on answering **queries** on a KG!
The notation will be detailed next.



One-hop Queries



Path Queries

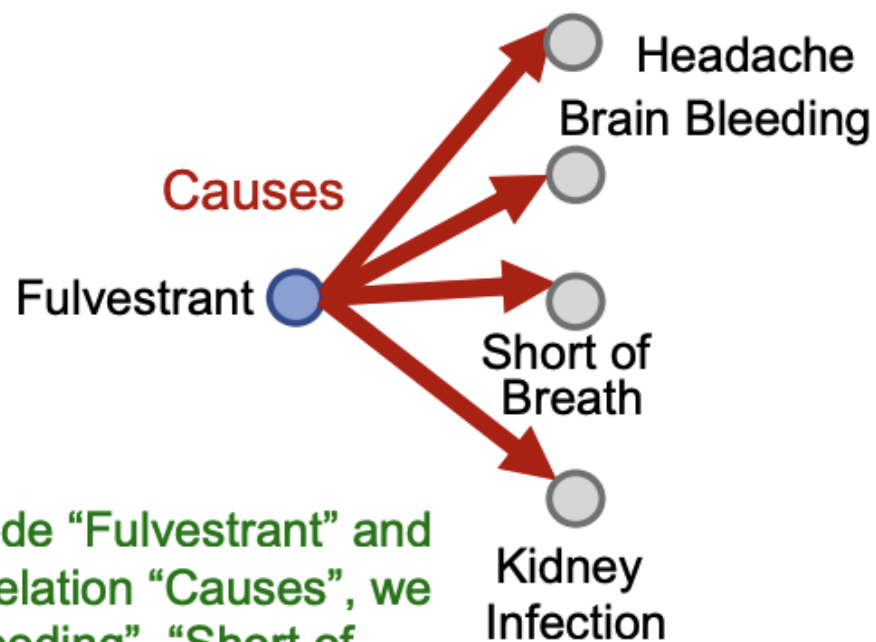


Conjunctive Queries



One-hop queries

- **One-hop query**: Is t an answer to the query (h, r) ?
- Example: What side effects are caused by the drug **Fulvestrant**?
- Query: $(e: \text{Fulvestrant}, (r: \text{Causes}))$



Start from the anchor node “Fulvestrant” and traverse the KG by the relation “Causes”, we reach entities {“Brain Bleeding”, “Short of Breath”, “Kidney Infection”, “Headache”}.



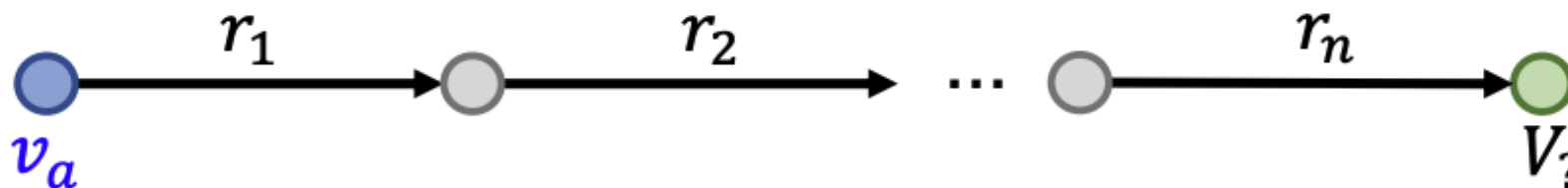
Path queries

- Generalize one-hop queries to path queries by adding more relations to the path

- An n -hop path query, q , can be represented by

$$q = (v_a, (r_1, \dots, r_n))$$

- v_a is an *anchor* entity
- Let answers to q in graph G be denoted by $\llbracket q \rrbracket_G$
- Query plan for q is a chain (as are all path queries)

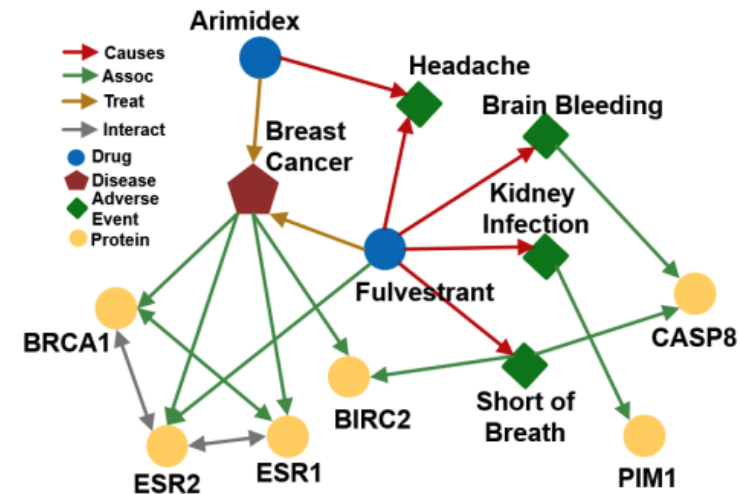
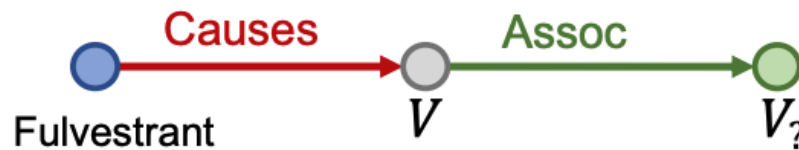




Path queries

Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?

- v_a is e:Fulvestrant
- (r_1, r_2) is (r:Causes, r:Assoc)
- Query: (e:Fulvestrant, (r:Causes, r:Assoc))

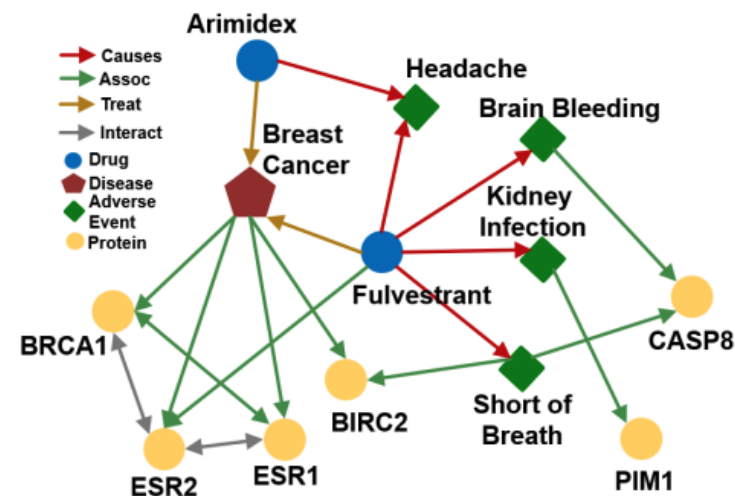
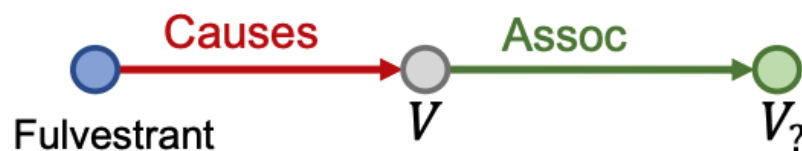




Path queries

Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?

- Query: (e:Fulvestrant, (r:Causes, r:Assoc))
- Given a KG, how do we answer a path query?





Traversing knowledge graphs for path queries

Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?

- Query: (e:Fulvestrant, (r:Causes, r:Assoc))

Fulvestrant 

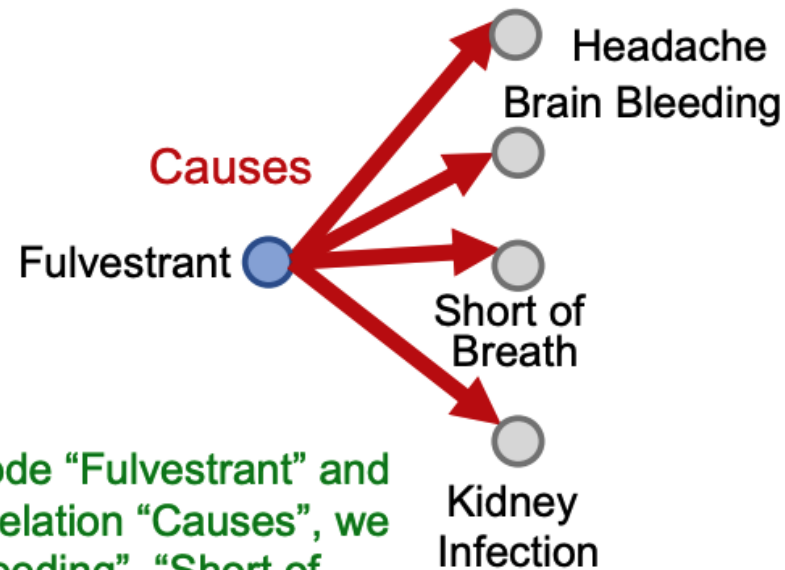
Start from the
anchor node
(Fulvestrant).



Traversing knowledge graphs for path queries

Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?

- Query: (e:Fulvestrant, (r:Causes, r:Assoc))



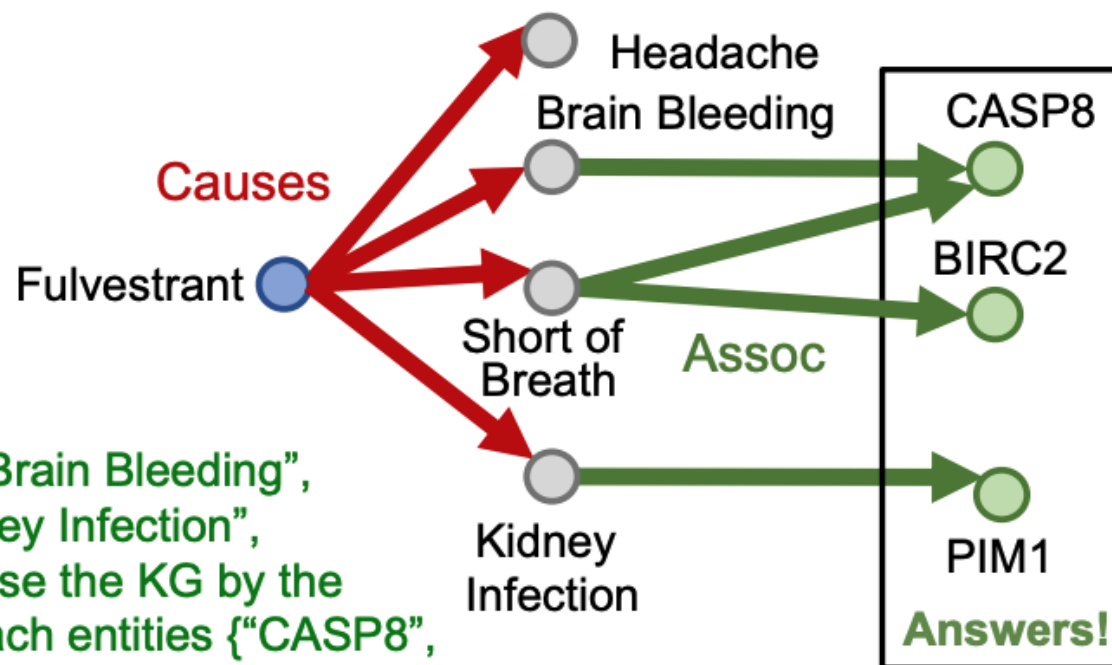
Start from the anchor node “Fulvestrant” and traverse the KG by the relation “Causes”, we reach entities {“Brain Bleeding”, “Short of Breath”, “Kidney Infection”, “Headache”}.



Traversing knowledge graphs for path queries

Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?

- Query: (e:Fulvestrant, (r:Causes, r:Assoc))



Start from the nodes {"Brain Bleeding", "Short of Breath", "Kidney Infection", "Headache"} and traverse the KG by the relation "Assoc", we reach entities {"CASP8", "BIRC2", "PIM1"}. These are the answers.

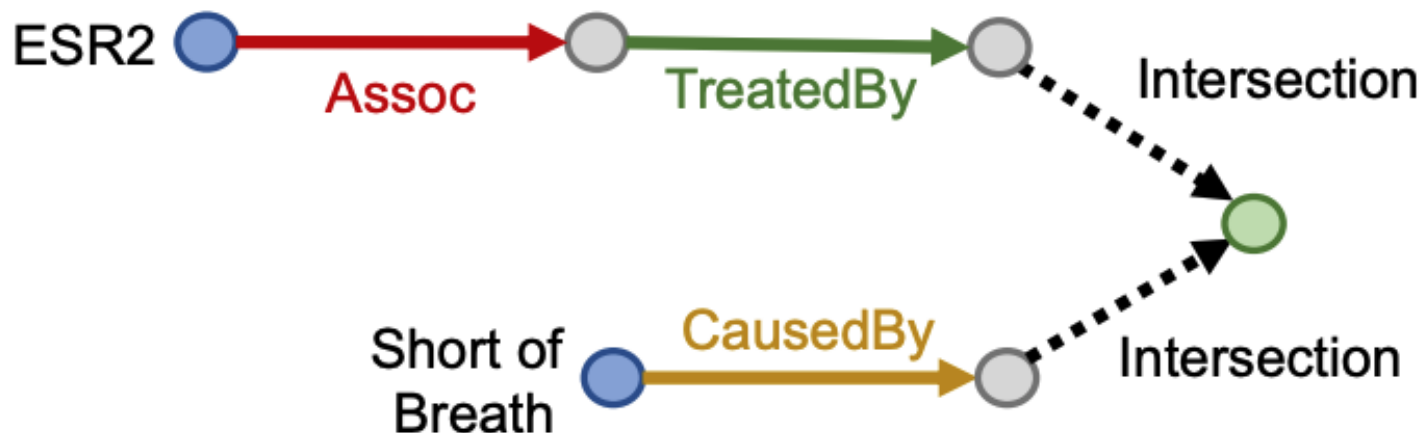


Conjunctive queries

Question: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Query: [(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short of Breath, (r:CausedBy))]

Query plan:

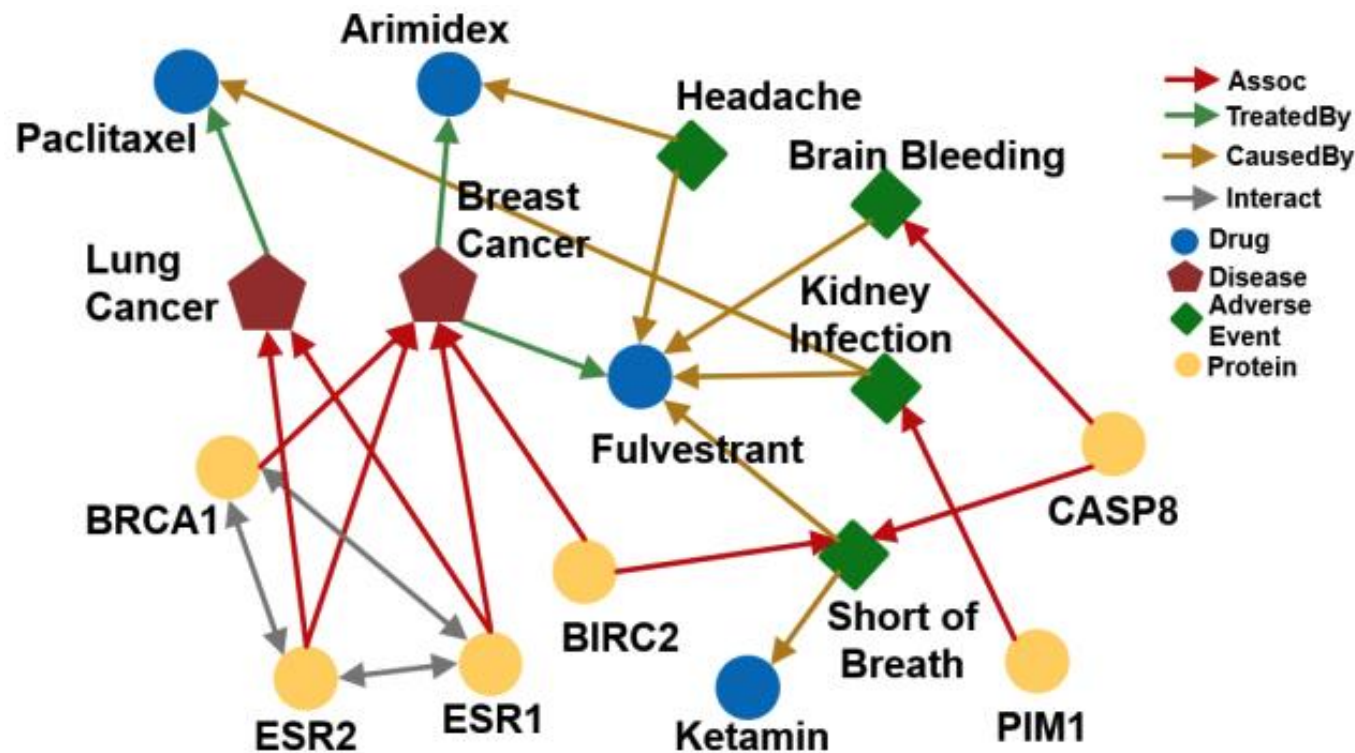




Conjunctive queries

Question: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Query: $[(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short\ of\ Breath, (r:CausedBy))]$
- How do we answer this question by KG traversal?

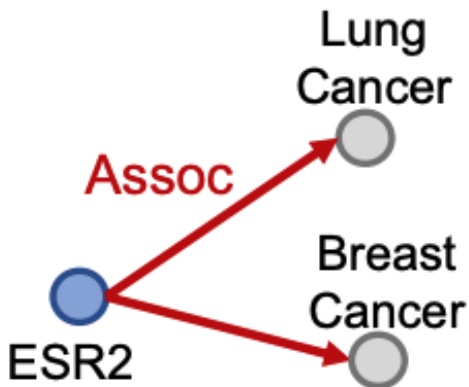




Traversing knowledge graphs for conjunctive queries

Question: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- **Query:** [(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short of Breath, (r:CausedBy))]
- Traverse KG from **anchor nodes:** ESR2 and Short of Breath



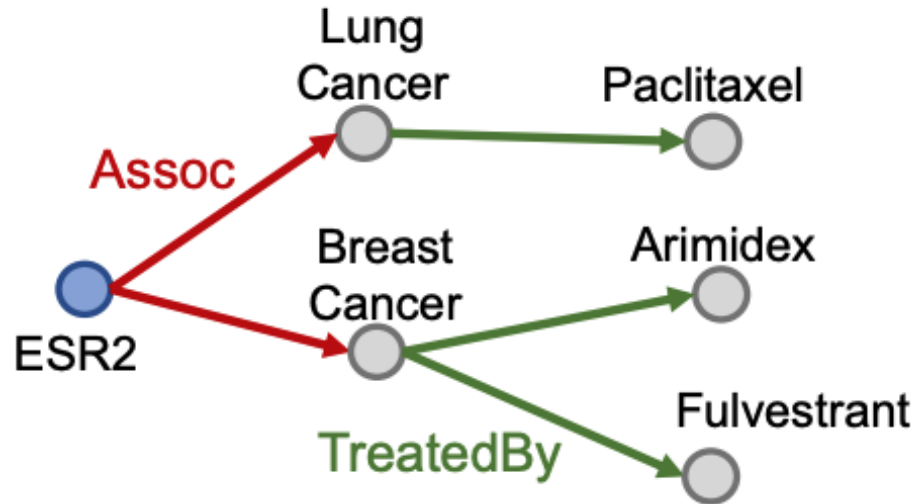
Traverse from the first anchor
ESR2 by relation *Assoc* to
reach entities {*Lung Cancer*,
Breast Cancer}



Traversing knowledge graphs for conjunctive queries

Question: What are drugs that *cause shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Query: $[(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short of Breath, (r:CausedBy))]$
- Traverse KG from anchor nodes: ESR2 and Short of Breath



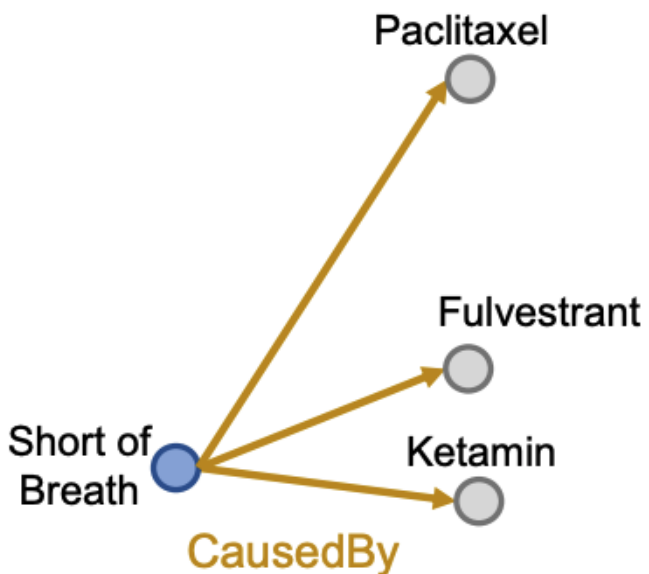
Traverse from the entities $\{Lung\ Cancer, Breast\ Cancer\}$ by the relation *TreatedBy* to reach the drug entities $\{Paclitaxel, Arimidex, Fulvestrant\}$



Traversing knowledge graphs for conjunctive queries

Question: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Query: [(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short of Breath, (r:CausedBy))]
- Traverse KG from anchor nodes: ESR2 and Short of Breath



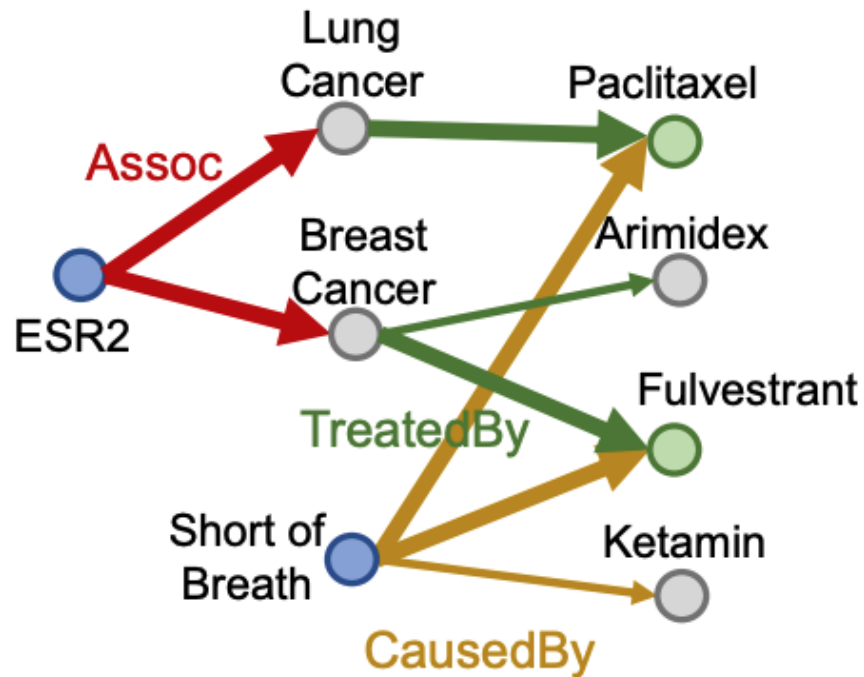
Traverse from the second anchor *Short of Breath* by the relation *CausedBy* to reach the set of entities {*Paclitaxel*, *Fulvestrant*, *Ketamin*}



Traversing knowledge graphs for conjunctive queries

Question: What are drugs that *cause shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- **Query:** [(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short of Breath, (r:CausedBy))]
- Traverse KG from **anchor nodes:** ESR2 and Short of Breath



Take the intersection between the two drug sets to get the answer set {*Paclitaxel*, *Fulvestrant*}



Predictive queries





Answering queries on incomplete KGs

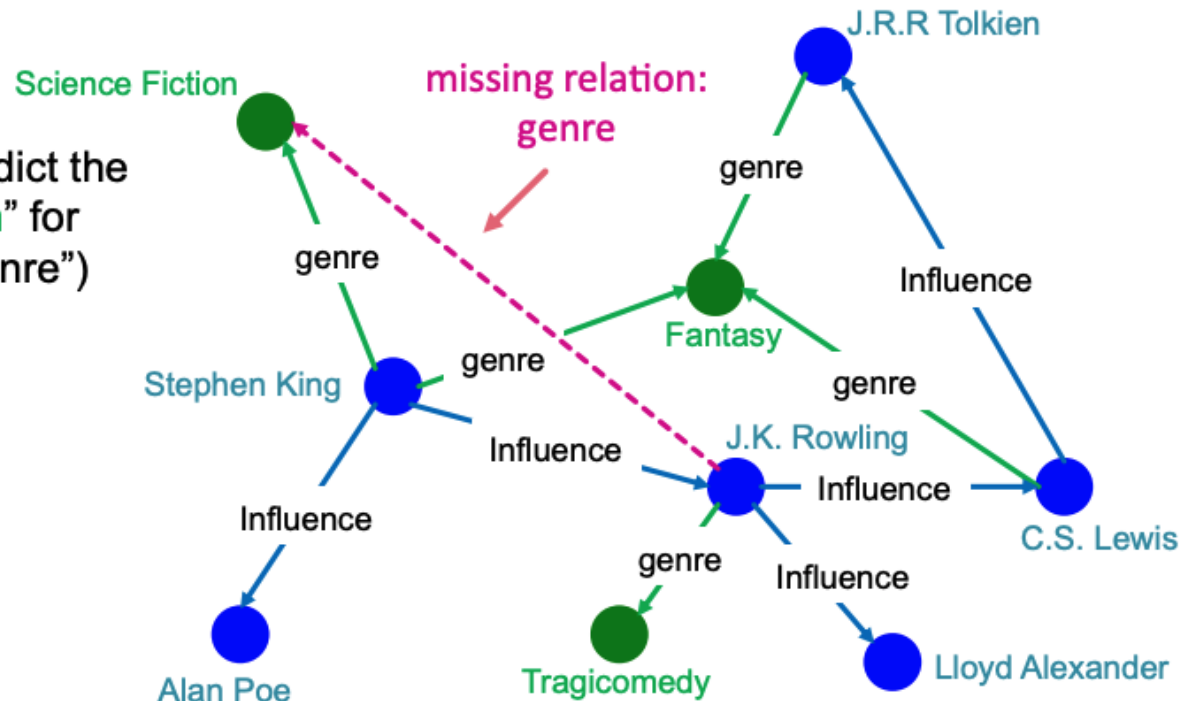
- Answering queries on KGs seems easy: just traverse the graph
- But KGs are incomplete
 - Many relations between entities are missing – for example, we lack knowledge of many associations among biomedical entities
 - Enumerating all facts requires non-trivial time and cost, so we shouldn't expect any KG to ever be complete
- Due to KG incompleteness, traversal alone will not identify all relations
 - Is this the best we can do?



Knowledge graph (KG) completion task

- Given a large KG, can we "complete" it (i.e., add missing edges)
- Task is typically formulated as predicting tails (or consequents)
 - Consider relations in a directed KG represented as a tuple (head, relation, tail) where head (h) has relation (r) with tail (t)
 - Given (head, relation) predict missing tails

Example task: predict the tail "Science Fiction" for ("J.K. Rowling", "genre")





Predictive one-hop queries

- We can formulate KG completion problems as answering one-hop queries
- **KG completion**: Is the edge (h, r, t) in the KG?



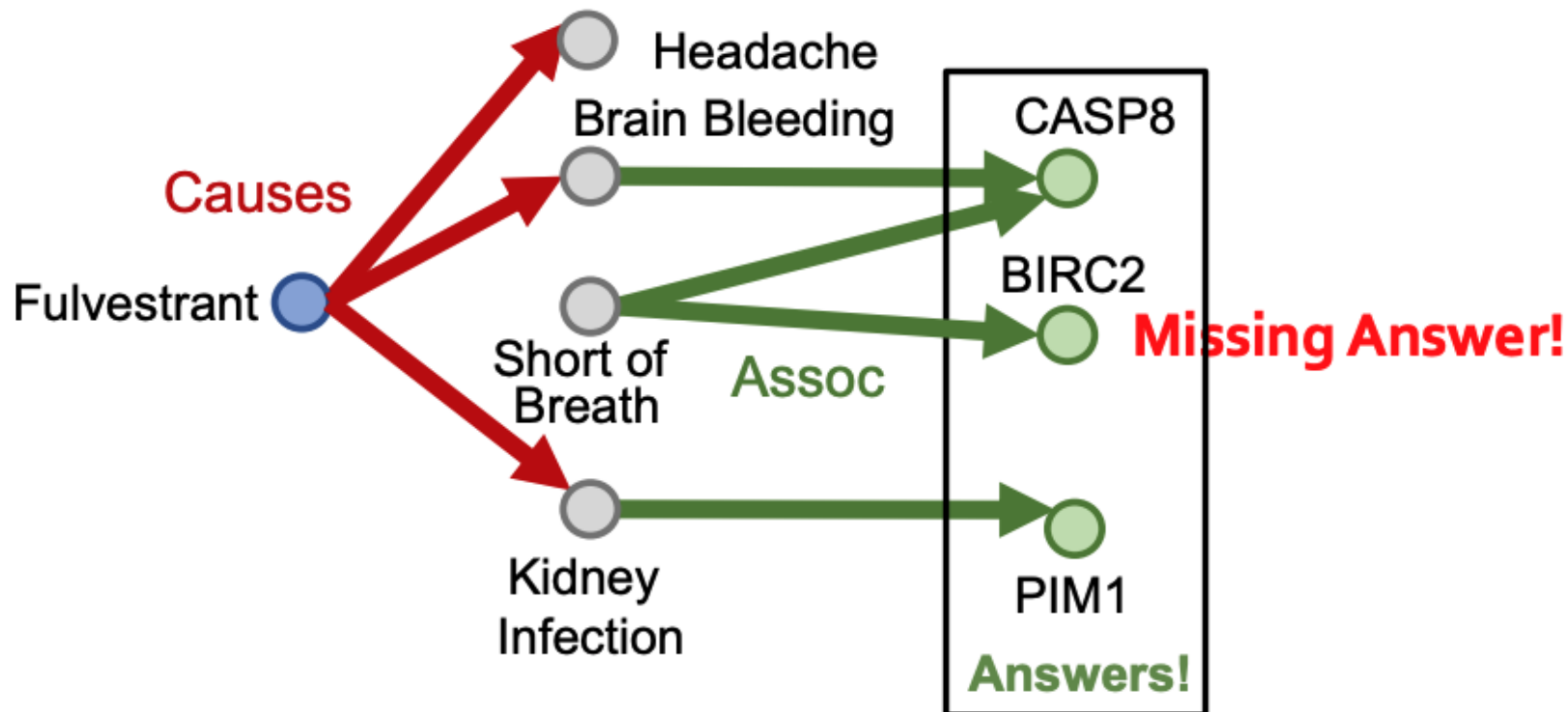
- **One-hop query**: Is t an answer to the query (h, r) ?
 - Example: What side effects are caused by the drug **Fulvestrant**?



Predictive path queries?

Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?

- Query: (e:Fulvestrant, (r:Causes, r:Assoc))

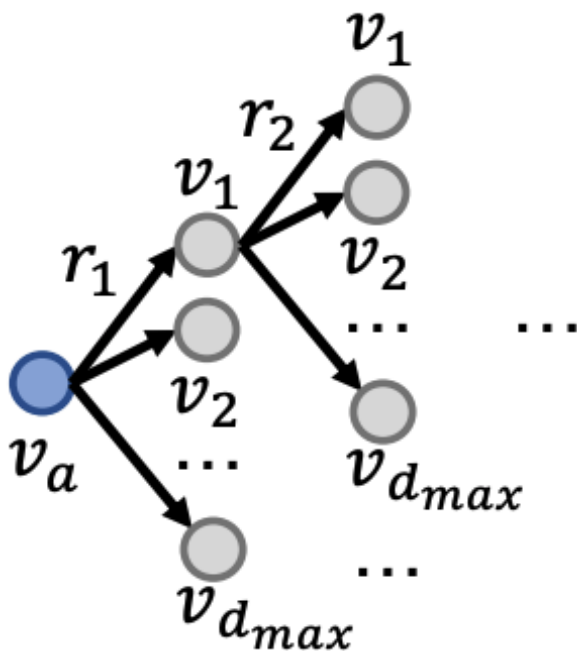




Predictive path queries

Can we first perform KG completion on the entire KG and then traverse the completed KG?

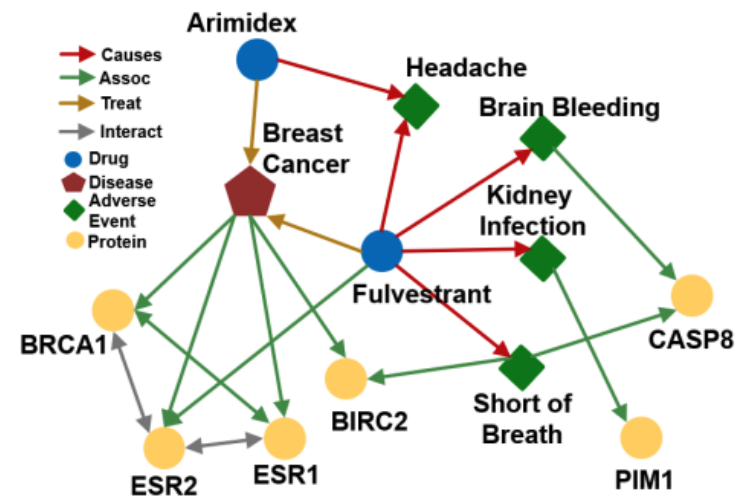
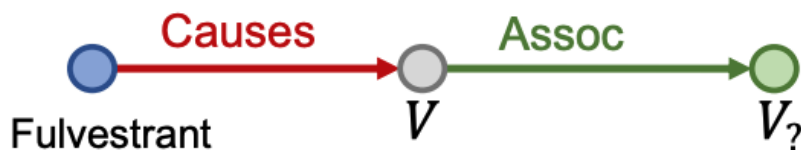
- No! The “completed” KG is a dense graph because most (h, r, t) triples will have some non-zero probability
- Time complexity of traversing a dense KG is $O(d_{max}^L)$ exponential with respect to the path length L
- What else can we do?





Predictive queries

- We need methods to answer path-based queries on an incomplete KG
 - Want to implicitly impute and account for missing relations (incompleteness)
 - Want to be able to answer arbitrary queries
- Generalization of the link prediction task

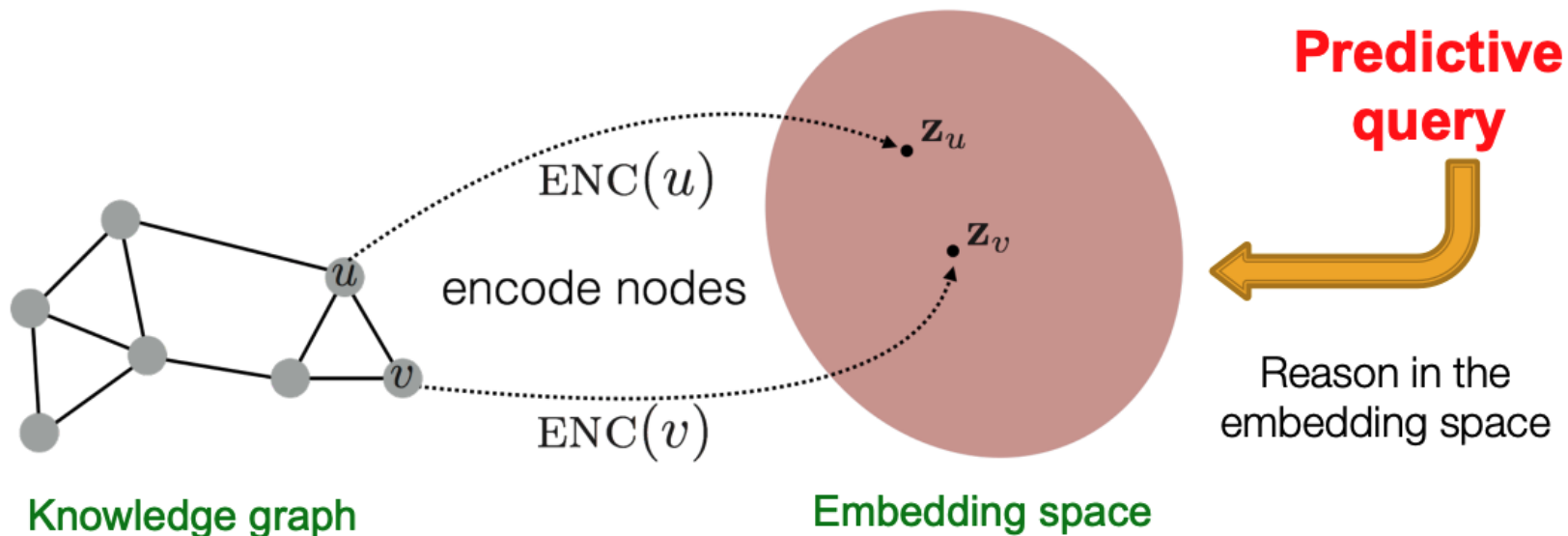




Embedding queries concept

Idea: Map queries into an embedding space and **learn to reason in that space**

- Embed query into a single **point** in the Euclidean space under the assumption that answer nodes are close to the query
- **Query2Box**: Embed query into a hyper-rectangle (**box**) in the Euclidean space under the assumption that answer nodes are in the box volume



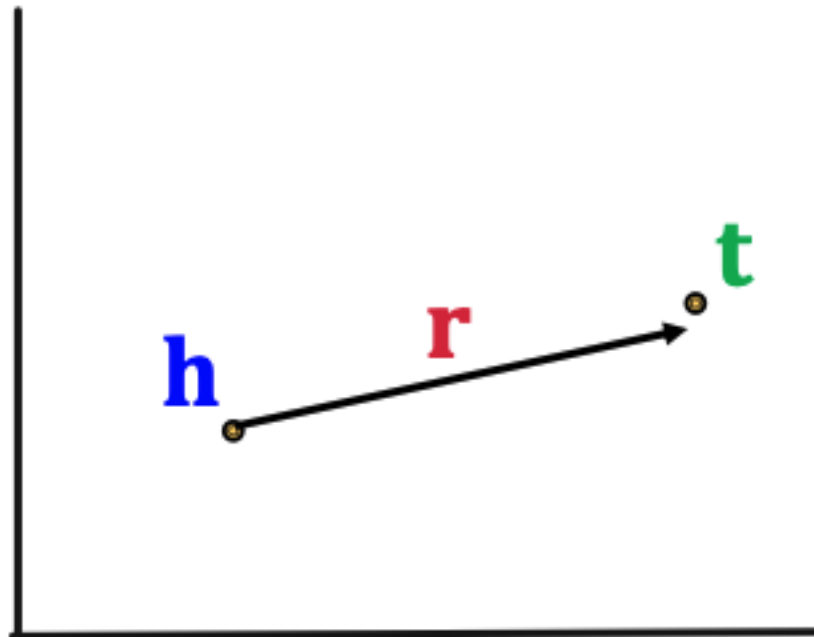


Predictive path queries with TransE



Trans(lation) Embedding (TransE)

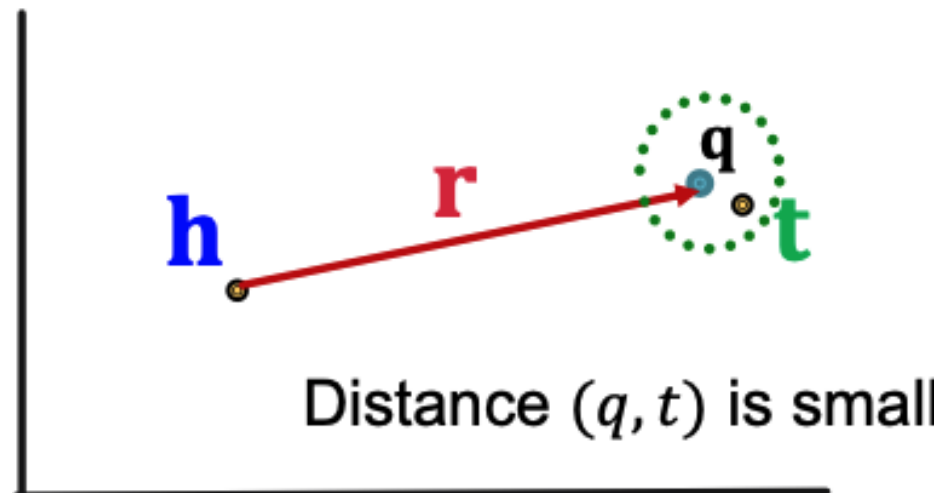
- Let $\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{R}^k$ be embeddings for the relation (h, r, t)
- **TransE**: $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ if the relation (h, r, t) exists, else $\mathbf{h} + \mathbf{r} \neq \mathbf{t}$
- Define the scoring function as $f_r(h, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|$
- Geometric interpretation as vectors will help us examine the ability of **TransE** to model different logic relations



Predictive path queries with TransE

- Generalize TransE to path queries
- TransE: Translates $\mathbf{h}$ to $\mathbf{t}$ using $\mathbf{r}$ with score function $f_r(h, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|$
- Alternative interpretation
 - **Query embedding**: $\mathbf{q} = \mathbf{h} + \mathbf{r}$
 - Goal: move $\mathbf{q}$ close to the **answer embedding** $\mathbf{t}$

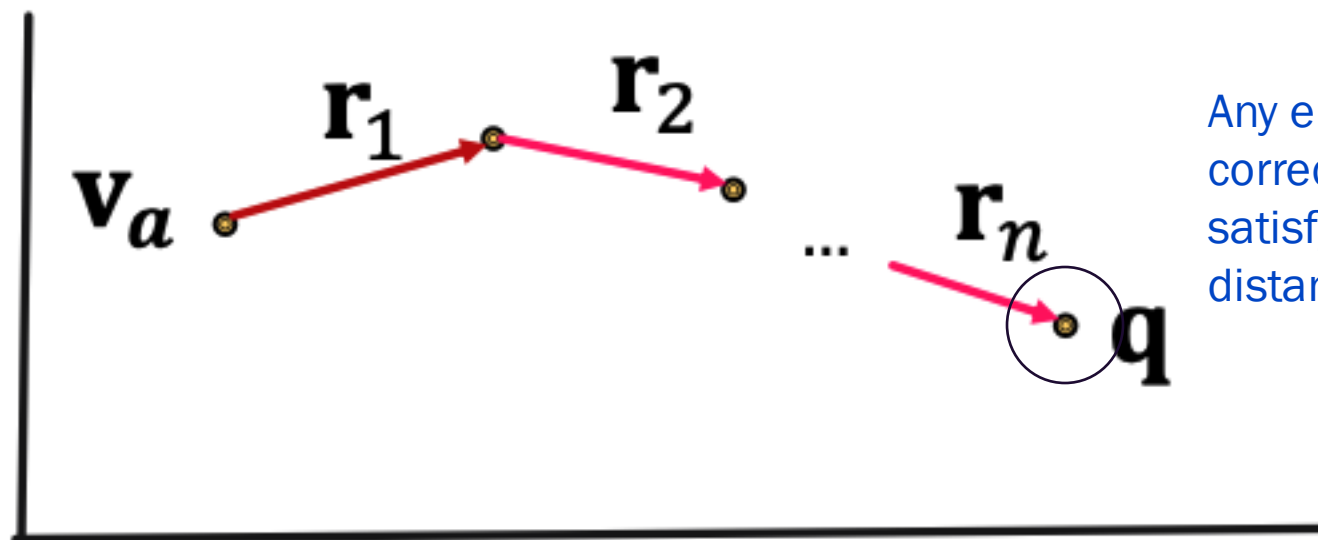
$$f_q(t) = -\|\mathbf{q} - \mathbf{t}\|$$



Predictive path queries with TransE

- Generalize TransE to path queries
- Given a path query : $q = (v_a, (r_1, \dots, r_n))$, set the query embedding to

$$\mathbf{q} = \mathbf{v}_a + \mathbf{r}_1 + \dots + \mathbf{r}_n$$
- The embedding process only involves vector addition that is independent of the number of entities in the KG



Any entity embeddings of the correct type (i.e., those that satisfy the query) within $\epsilon > 0$ distance of $\mathbf{q}$



An example of predictive path queries with TransE

- Question: *What proteins are **associated** with adverse events **caused** by **Fulvestrant**?*
- Query: (e:Fulvestrant, (r:Causes, r:Assoc))

Follow the query plan:

Query Plan

Fulvestrant 

Embedding Process

Fulvestrant 



An example of predictive path queries with TransE

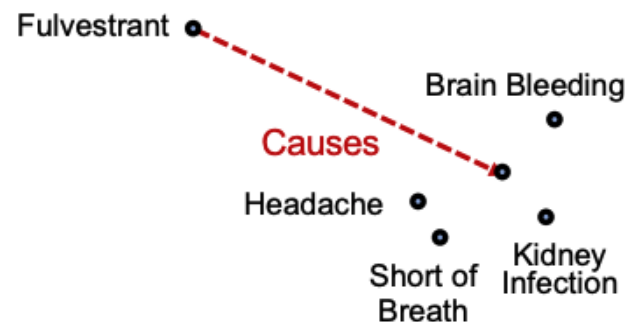
- Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?
- Query: (e:Fulvestrant, (r:Causes, r:Assoc))

Follow the query plan:

Query Plan



Embedding Process



An example of predictive path queries with TransE

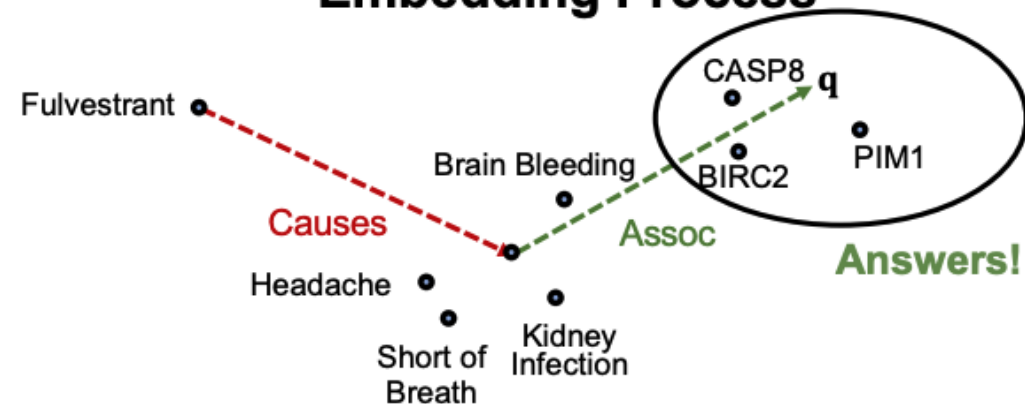
- Question: What proteins are *associated* with adverse events *caused* by *Fulvestrant*?
- Query: (e:Fulvestrant, (r:Causes, r:Assoc))

Follow the query plan:

Query Plan



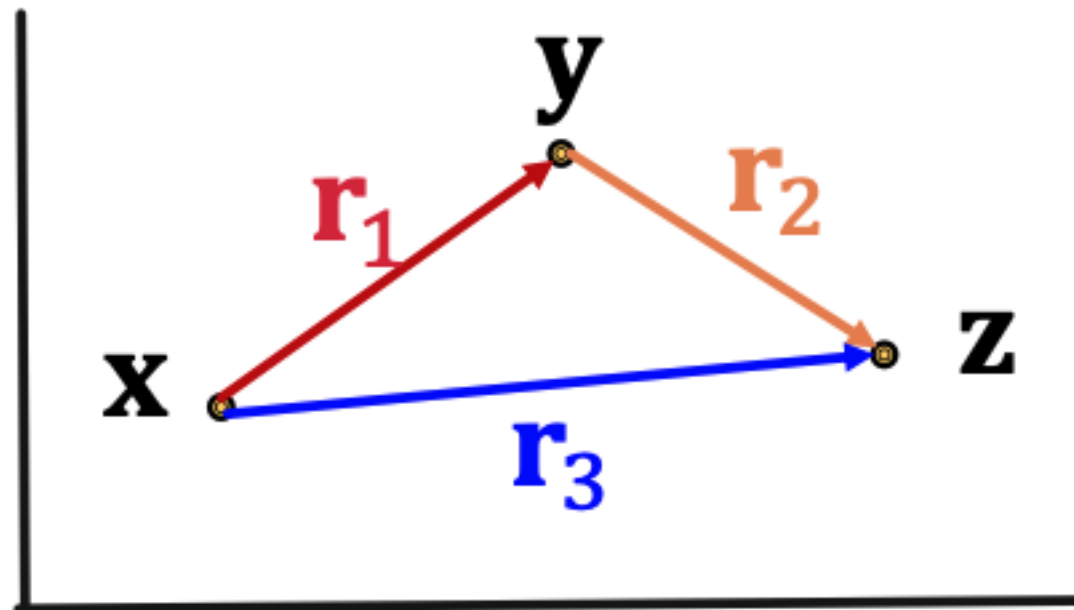
Embedding Process





Predictive path query training on TransE

- We can train a TransE model to optimize the knowledge graph completion objective (tail completion)
- Recall, TransE handles compositional relations, hence it naturally handles path queries over multiple hops
 - We get this for “free”!





Predictive conjunctive queries with Query2Box

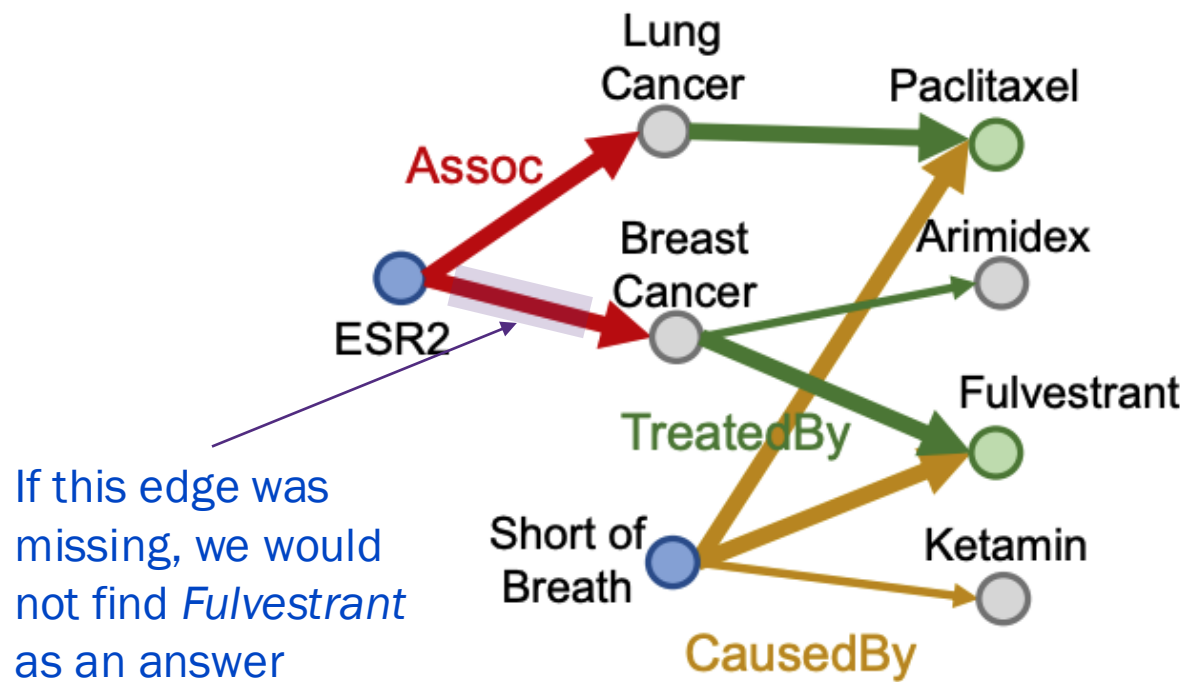




Predictive conjunctive queries

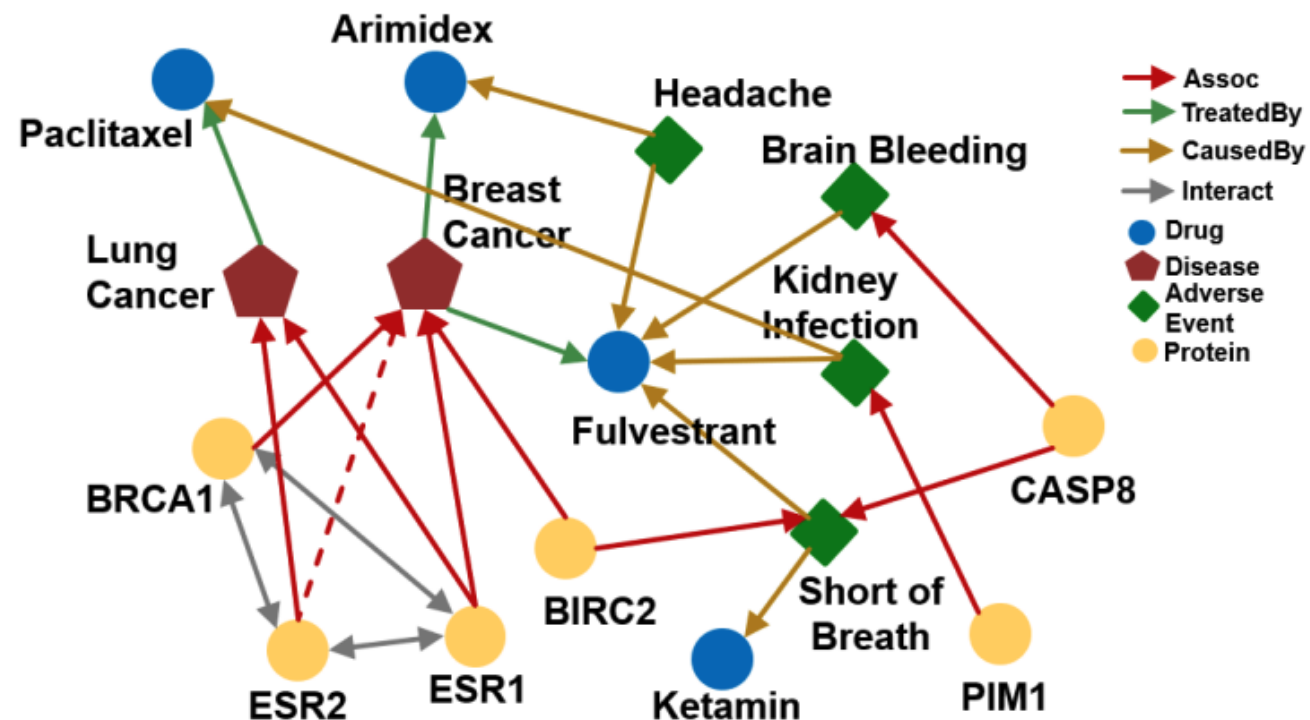
Question: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Query: $[(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short\ of\ Breath, (r:CausedBy))]$
- Traverse KG from anchor nodes: *ESR2* and *Short of Breath*



Predictive conjunctive queries

- How can we use embeddings to implicitly impute the missing relation (*ESR2, Assoc, Breast Cancer*)?
- Intuition: ESR2 interacts with BRCA1 and ESR1, both of which are associated with breast cancer.





Box embeddings

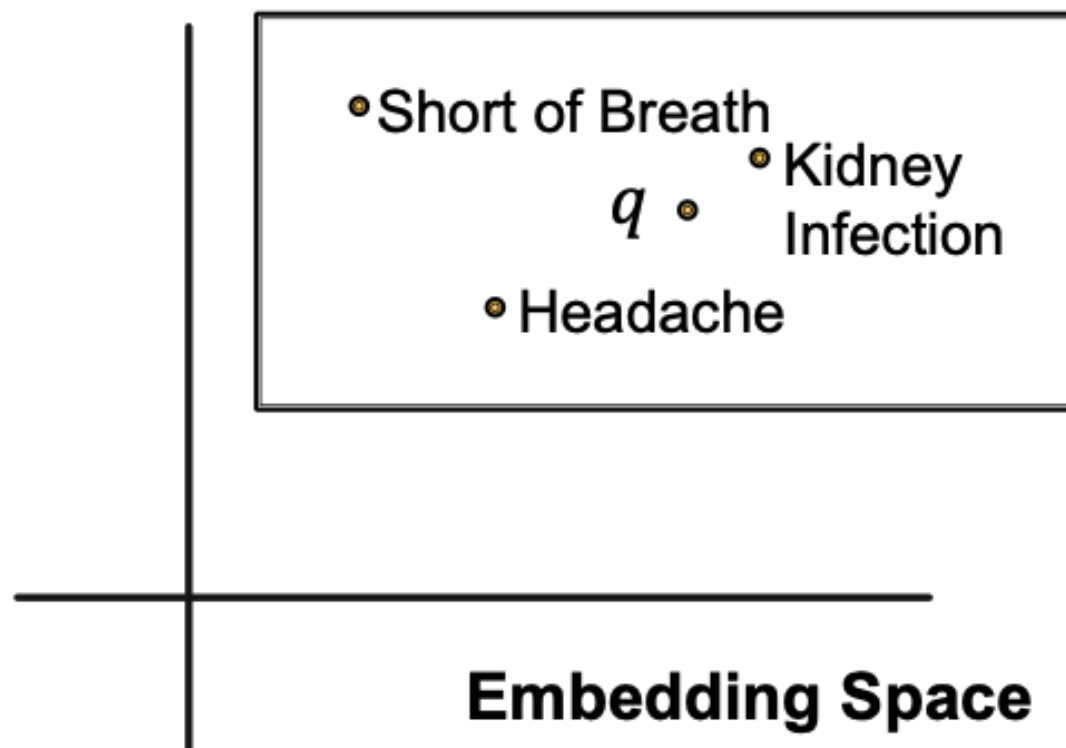
- Embed queries, q , in a hyper-rectangle (box)

$$\mathbf{q} \equiv \{\mathbf{c}_q, \mathbf{o}_q\}$$

$\mathbf{q} \in \mathbb{R}^{2d}$ is the box

$\mathbf{c}_q \in \mathbb{R}^d$ center of box

$\mathbf{o}_q \in \mathbb{R}^d$ offset from center



Example: Embed the adverse events of Fulvestrant within a box that encloses all answer entities.



Box embeddings

Some things we need to figure out:

- Entity embeddings (# parameters: $d|V|$)
 - Individual entities are *zero-volume* boxes (i.e., points)
- Relation embeddings (# parameters: $2d|R|$)
 - Each relation maps a box to a new box
- Intersection operator $\mathcal{J}$
 - Maps input boxes to output boxes
 - Models intersection of boxes

d : embedding dimension
 $|V|$: number of entities (nodes)
 $|R|$: number of relations



Embed conjunctive queries

Embed queries in vector space: *What are drugs that **cause** **shortness of breath** and **treat** diseases **associated** with protein **ESR2**?*

- **Query:** [(e:ESR2, (r:Assoc, r:TreatedBy)), (e:Short of Breath, (r:CausedBy))]
- Traverse KG from **anchor nodes:** ESR2 and Short of Breath

Query plan



Embedding Space

**How should we
do this?**

ESR2 •



Projection operator

- Idea: Take the current box as input and use the **relation embedding** to project and expand the box

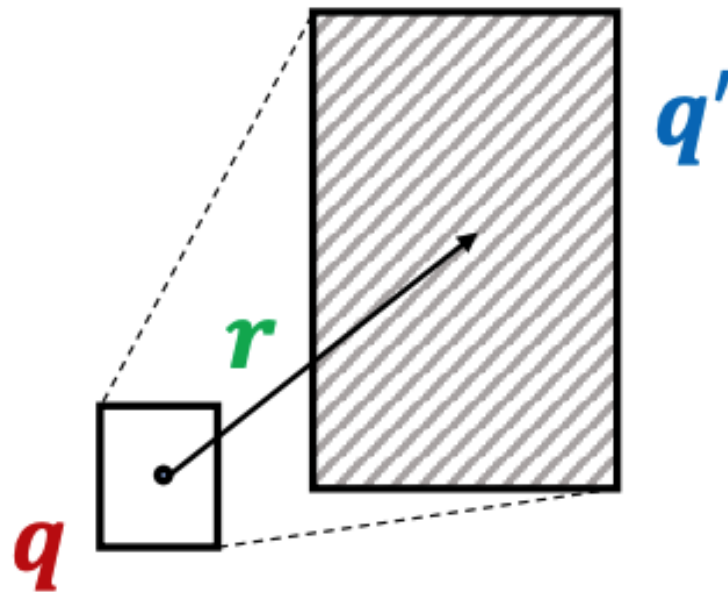
- Projection operator $\mathcal{P}: Box \times Relation \rightarrow Box$ $\times$ means the projection operator is a mapping from an input box and relation to an output box

$\mathbf{c}_q \in \mathbb{R}^d$ center of box

$$\mathbf{c}_{q'} = \mathbf{c}_q + \mathbf{c}_r$$

$\mathbf{o}_q \in \mathbb{R}^d$ offset from center

$$\mathbf{o}_{q'} = \mathbf{o}_q + \mathbf{o}_r$$



Implementation: we don't actually learn an operator. Rather, we learn two (2) embeddings for every relation type in the graph, $\mathbf{c}_r$ and $\mathbf{o}_r$ that we use to translate and expand boxes



Embed conjunctive queries

Embed queries in vector space: *What are drugs that **cause** **shortness of breath** and **treat** diseases **associated** with protein **ESR2**?*

- Traverse KG from **anchor nodes**: **ESR2** and **Short of Breath**
- Use the **projection operator** to follow the relations in the query plan

Query Plan



Embedding Space





Embed conjunctive queries

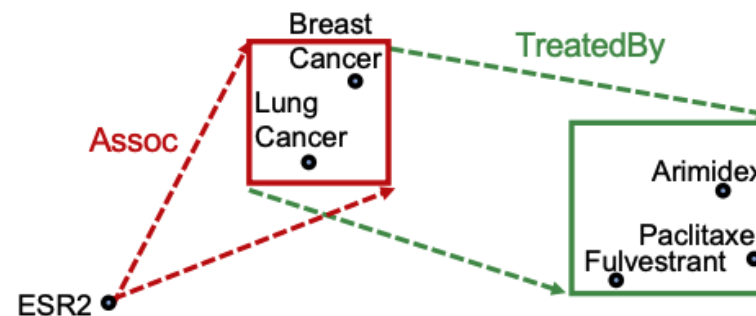
Embed queries in vector space: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Traverse KG from **anchor nodes**: *ESR2* and *Short of Breath*
- Use the **projection operator** to follow the relations in the query plan

Query Plan



Embedding Space



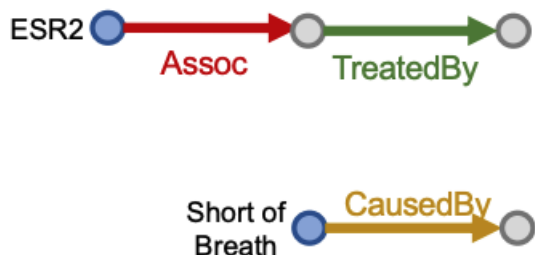


Embed conjunctive queries

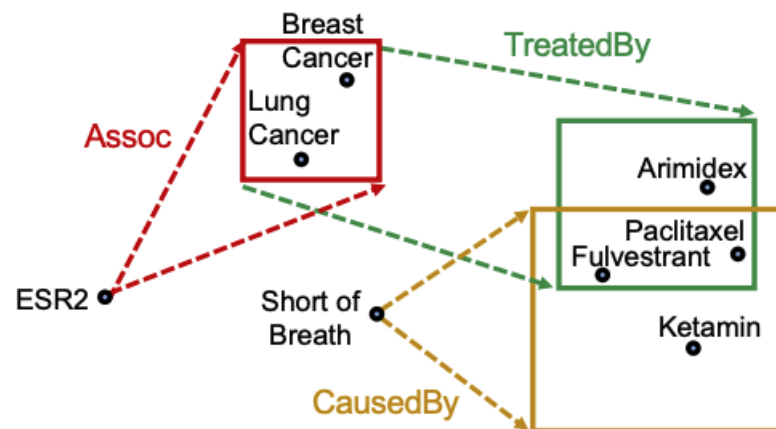
Embed queries in vector space: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Traverse KG from **anchor nodes**: **ESR2** and **Short of Breath**
- Use the **projection operator** to follow the relations in the query plan

Query Plan



Embedding Space



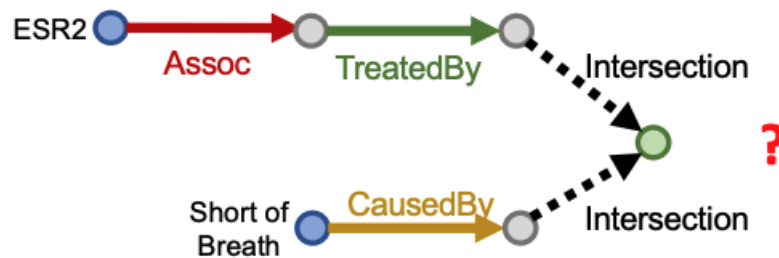


Embed conjunctive queries

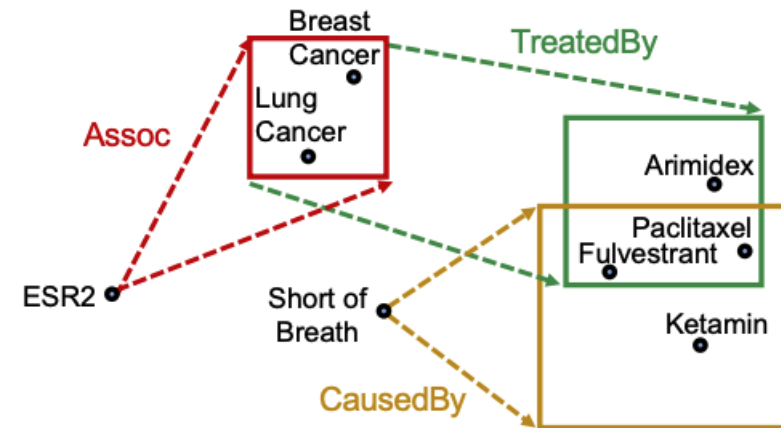
Embed queries in vector space: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- How do we take the intersection of the hyper-boxes?

Query Plan



Embedding Space



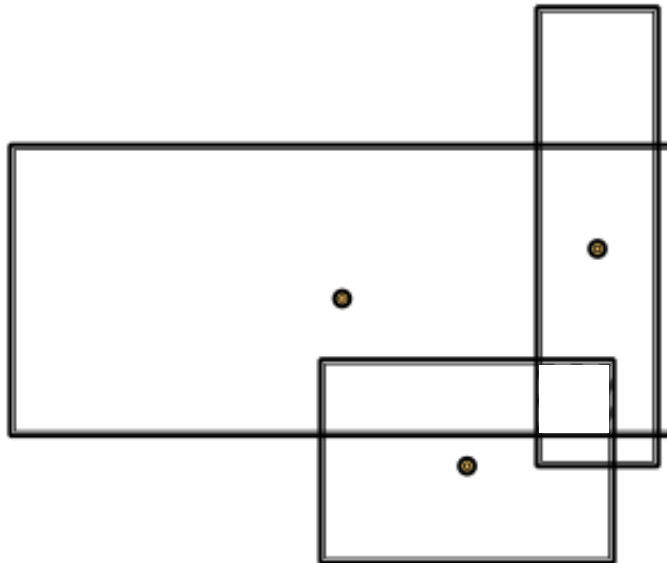
The answer to the query is the set of elements in the intersection of the hyper-boxes



Intersection operator

Intersection operator $\mathcal{J}: Box \times \dots \times Box \rightarrow Box$ is a mapping from multiple hyper-boxes to their intersection

- The center of the intersection should be “close” to the centers of the input boxes
- The offset (box size) should shrink since the size of the intersection should be smaller than the size of all the input sets





Intersection operator

Intersection operator $\mathcal{J}: \text{Box} \times \dots \times \text{Box} \rightarrow \text{Box}$

$$\mathbf{c}_{\mathbf{q}_{int}} = \sum_i \mathbf{w}_i \odot \mathbf{c}_{\mathbf{q}_i}$$

MLP is a neural network
with learnable parameters

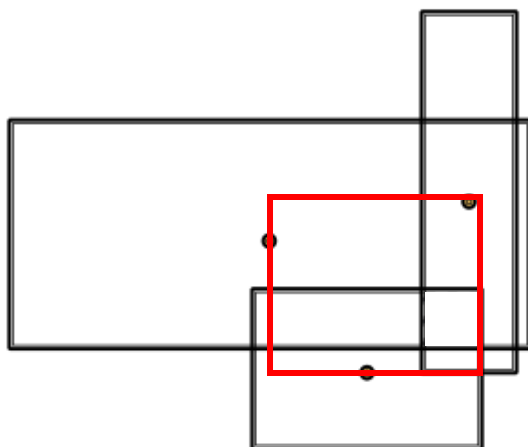
$$\mathbf{w}_i = \frac{\exp[MLP(\mathbf{q}_i)]}{\sum_j \exp[MLP(\mathbf{q}_j)]}$$

$\mathbf{q} \in \mathbb{R}^{2d}$ is the box

$\mathbf{c}_{\mathbf{q}} \in \mathbb{R}^d$ center of box

$\mathbf{o}_{\mathbf{q}} \in \mathbb{R}^d$ offset from center

$\odot$ is the Hadamard product (element
wise multiplication)



The center of the intersection is a weighted sum of the input
box centers and is in the red region

$\mathbf{w}_i$ are attention scores over the individual input box centers



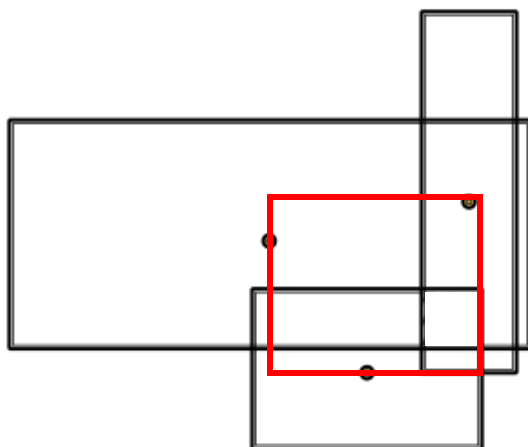
Intersection operator

Intersection operator $\mathcal{J}: Box \times \dots \times Box \rightarrow Box$

$$\mathbf{o}_{q_{int}} = \min_j \{ \mathbf{o}_{q_j} \} \odot \sigma[f_{\mathbf{o}}(\{\mathbf{q}_1, \dots, \mathbf{q}_j\})]$$

σ sigmoid,
guarantees
shrinking of the box

$f_{\mathbf{o}}$ is a neural network with learnable parameters
that operates on sets (e.g., DeepSets)



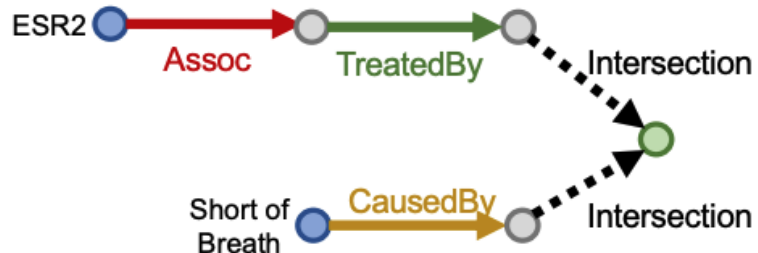
- The offset should be smaller than the offset of the input boxes but not necessarily equal to it
- Design a neural network, $f_{\mathbf{o}}$, to learn the appropriate scaling of the minimum input offset

Embed conjunctive queries

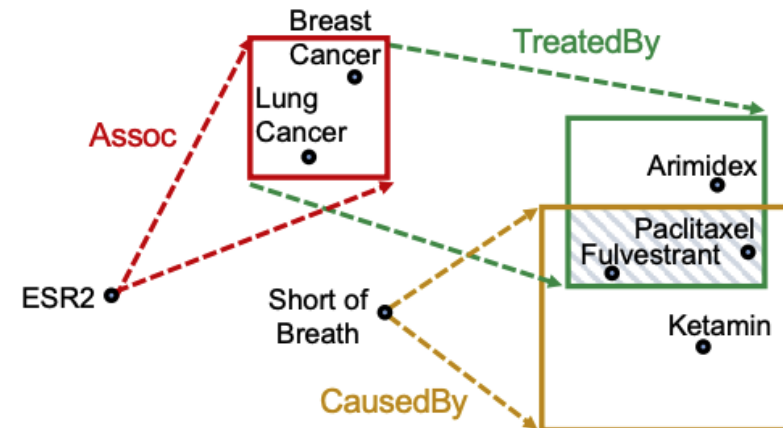
Embed queries in vector space: What are drugs that *cause* *shortness of breath* and *treat* diseases *associated* with protein *ESR2*?

- Combine the projection and box intersection operators

Query Plan



Embedding Space



The shaded box represents the final box (with answers) for the query embedding



Distance to the query box

- Which entity embeddings are in the query box?
- Consider a query box $\mathbf{q}$ and an entity embedding $\mathbf{v}$
- Define the distance between $\mathbf{q}$ and $\mathbf{v}$ by

$$\text{dist}_{\text{box}}(\mathbf{v}; \mathbf{q}) = \text{dist}_{\text{out}}(\mathbf{v}; \mathbf{q}) + \alpha \cdot \text{dist}_{\text{in}}(\mathbf{v}; \mathbf{q})$$

where $0 < \alpha < 1$ and

$$\text{dist}_{\text{out}}(\mathbf{v}; \mathbf{q}) = \|\max(\mathbf{v} - \mathbf{q}_{\max}, \mathbf{0}) + \max(\mathbf{q}_{\min} - \mathbf{v}, \mathbf{0})\|_1$$

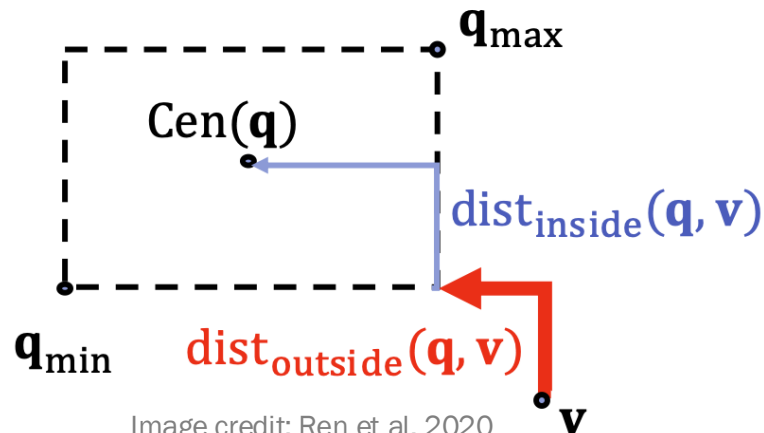
element wise max / min

$$\text{dist}_{\text{in}}(\mathbf{v}; \mathbf{q}) = \|\mathbf{c}_{\mathbf{q}} - \min(\mathbf{q}_{\max}, \max(\mathbf{q}_{\min}, \mathbf{v}))\|_1$$

$\|\mathbf{x}\|_1 = \sum_i |x_i|$

where $\mathbf{q}_{\max} = \mathbf{c}_{\mathbf{q}} + \mathbf{o}_{\mathbf{q}}$ and $\mathbf{q}_{\min} = \mathbf{c}_{\mathbf{q}} - \mathbf{o}_{\mathbf{q}}$

Setting $0 < \alpha$ downweights distance in the box, indicating entities are “close enough” to the query center





Training Query2Box

- What are the trainable parameters?
 - Entity embeddings (# parameters: $d|V|$)
 - Relation embeddings (# parameters: $2d|R|$)
 - Intersection operator $\mathcal{J}$ which contains two neural networks
- Loss function

$$\mathcal{L} = -\log[\sigma(\gamma - \text{dist}_{\text{box}}(\mathbf{v}; \mathbf{q}))] - \frac{1}{k} \sum_{i=1}^k \log[\sigma(\text{dist}_{\text{box}}(\mathbf{v}'_i; \mathbf{q}) - \gamma)]$$

where γ is a scalar margin, $v \in \llbracket q \rrbracket$ is a positive entity (i.e., correct answer to the query, q) and $v'_i \notin \llbracket q \rrbracket$ is the i -th negative entity (non-answer to q)



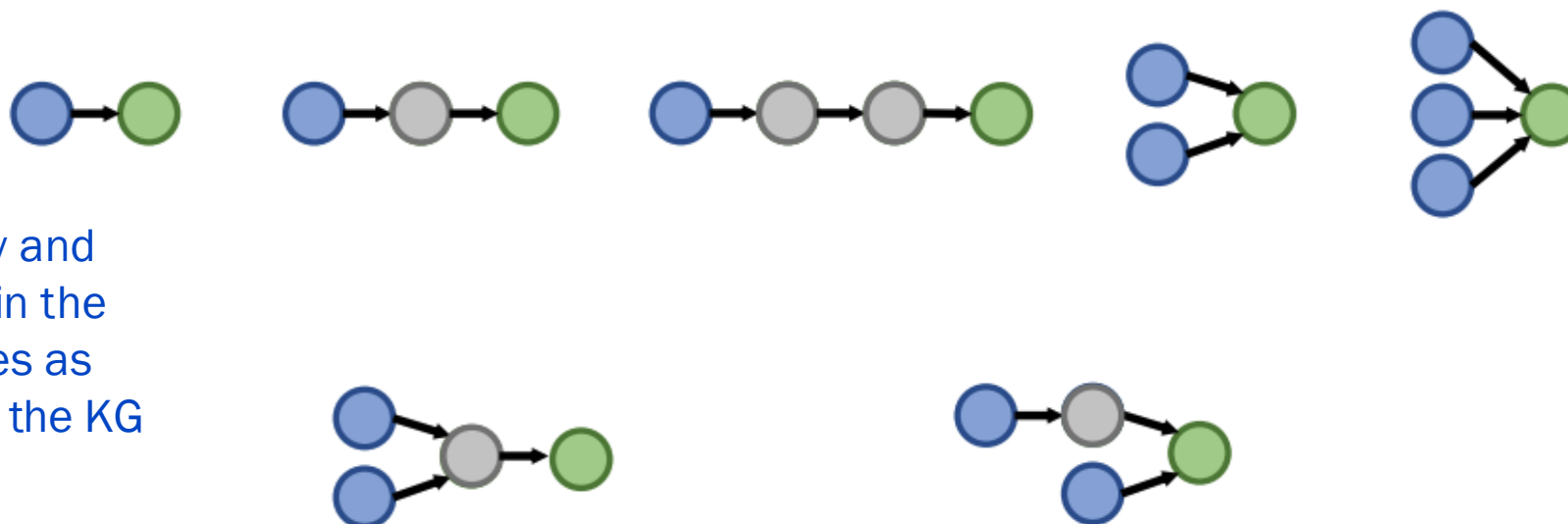
Training Query2Box

- Training loop
 1. Sample a query from training data, (q, v, v')
 2. Use current embeddings and intersection operator to embed q
 3. Compute loss and update embedding and intersection operator weights
- **How do we obtain training data and implement training?**



Generating sample queries on a KG

Start with query templates that represent possible query plans in the KG of interest

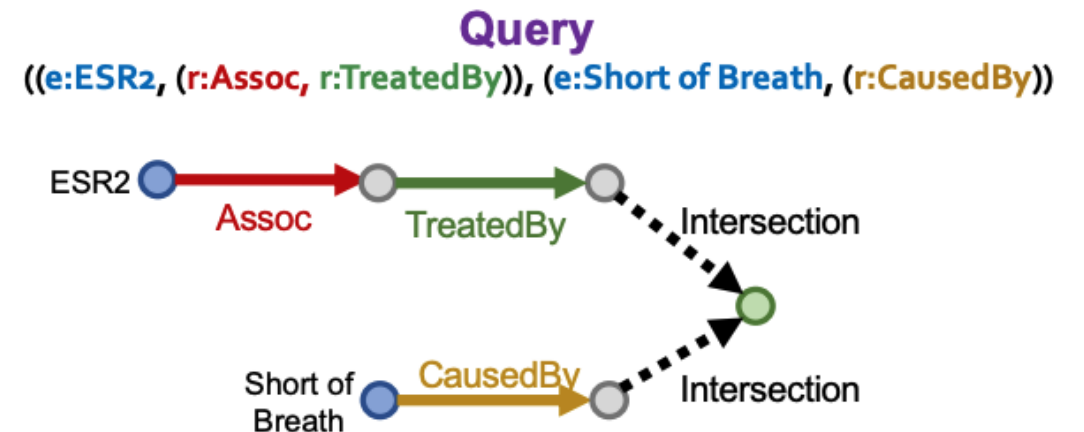
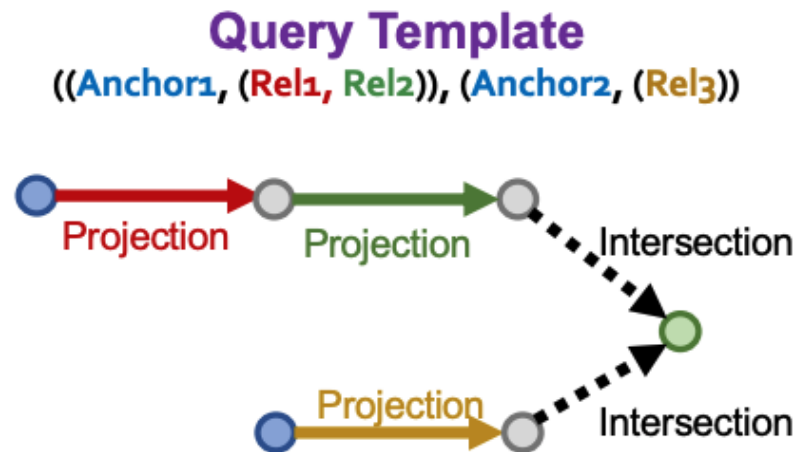


Different entity and relation types in the query templates as determined by the KG



Generating sample queries on a KG

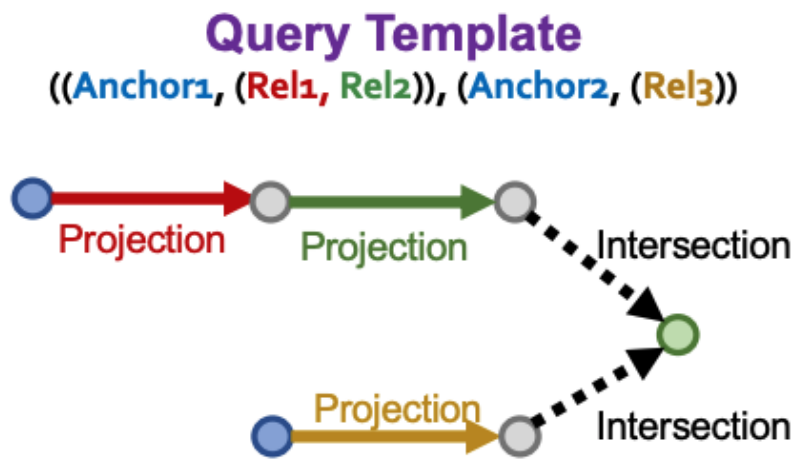
- Pick one of the query templates
- Generate sample queries by instantiating every variable with a concrete entity and relation from the KG
 - Example: instantiate Anchor1 with ESR2, Rel1 with Assoc
- How to do this systematically?



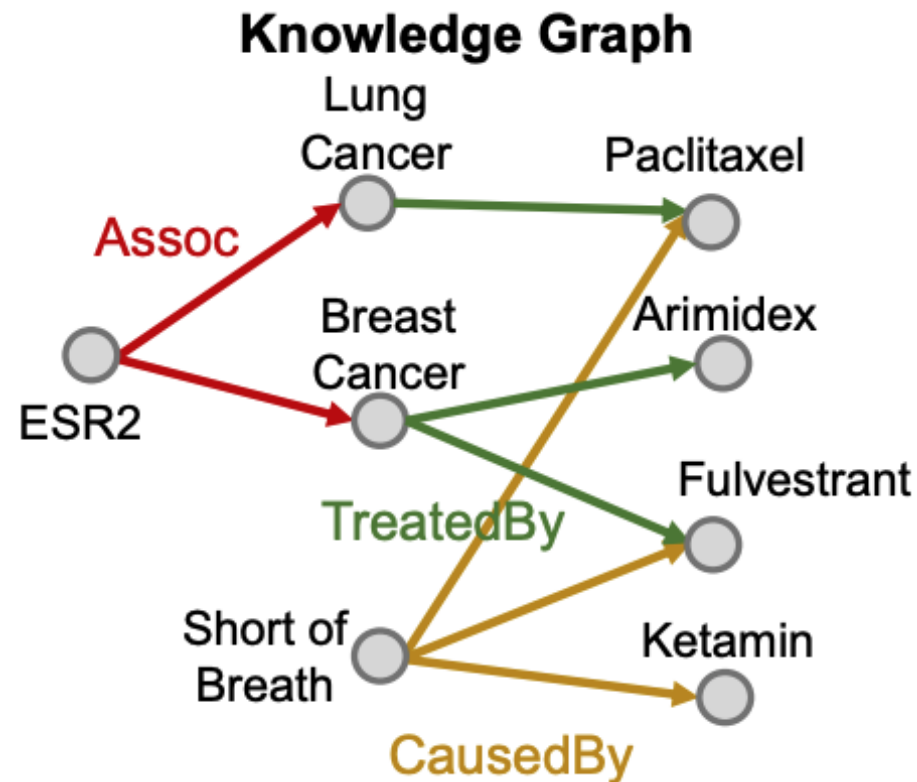


Generating sample queries on a KG

- How to do this systematically?

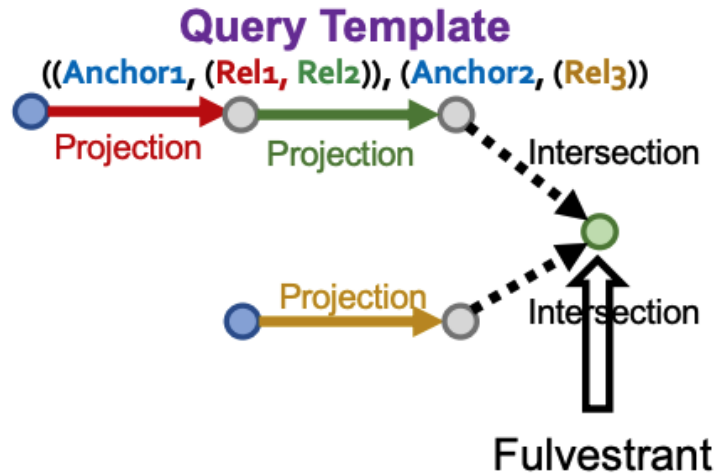


Overview: Start by instantiating an **answer node** of the query template and iteratively instantiate the other edges and nodes until we ground all anchor nodes



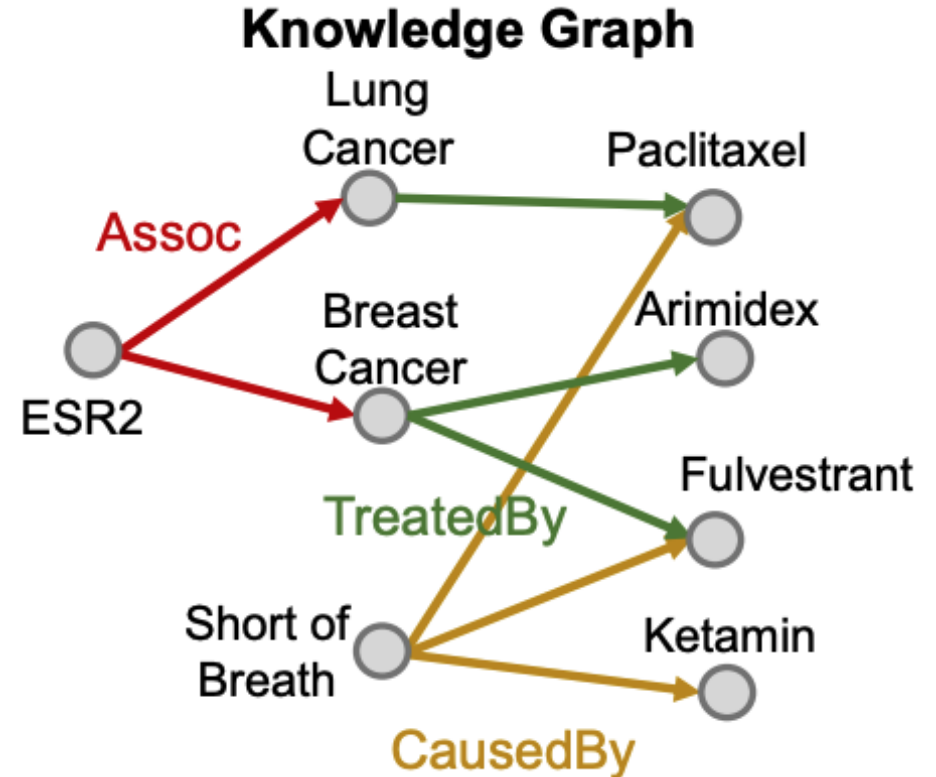
Generating sample queries on a KG

- How to do this systematically?



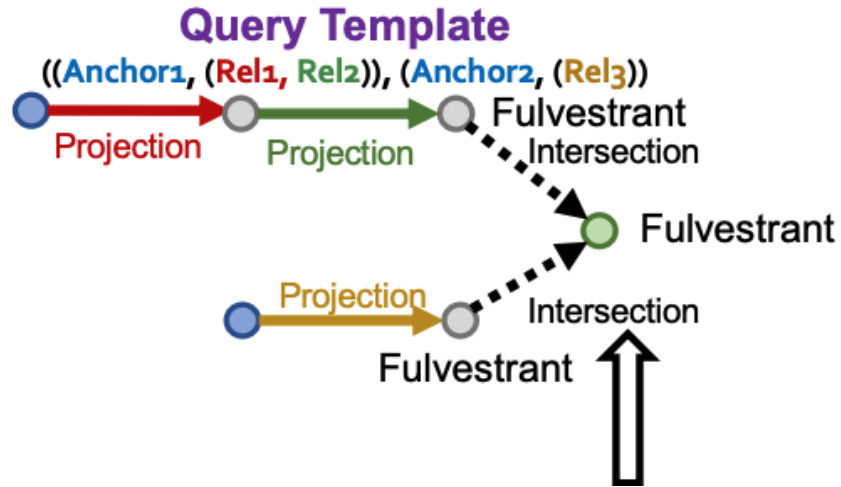
Start from the answer (root) of the query template.
Why? As we will encounter intersections, we want to ensure the preceding paths include the final answer (see next slide)

Randomly pick one entity from the KG as the root node – in this example, Fulvestrant

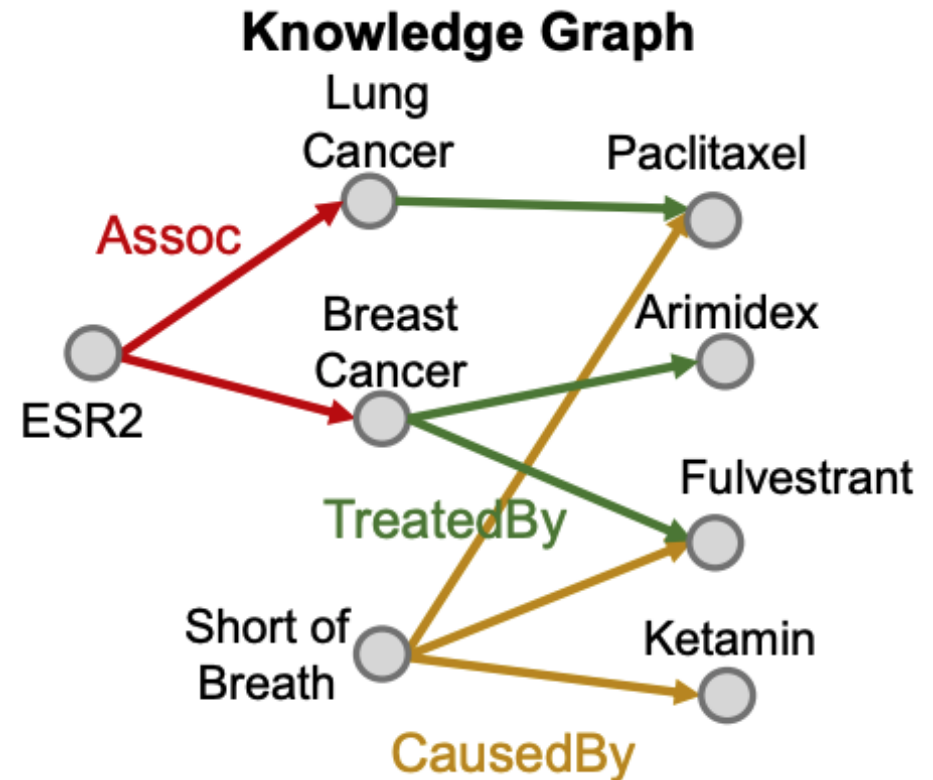


Generating sample queries on a KG

- How to do this systematically?

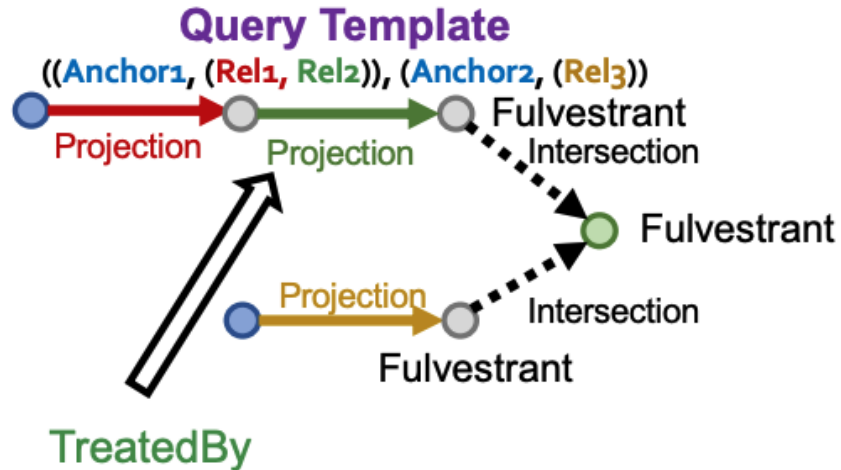


Since we are considering an intersection, we require that Fulvestrant is the entity ending the query paths at the preceding level in the query.

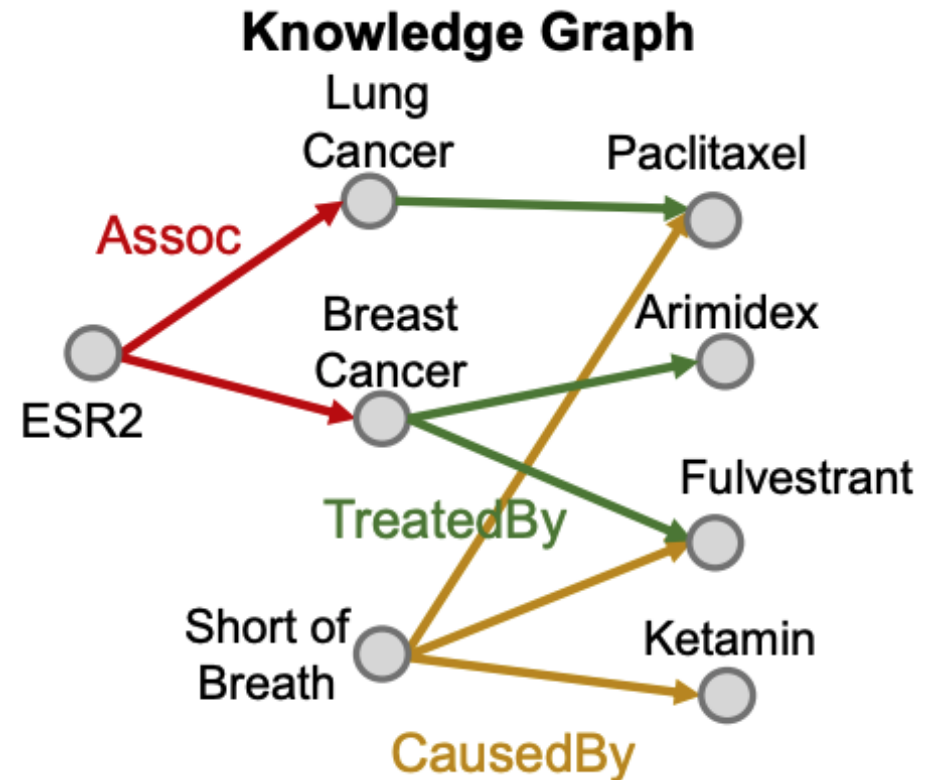


Generating sample queries on a KG

- How to do this systematically?



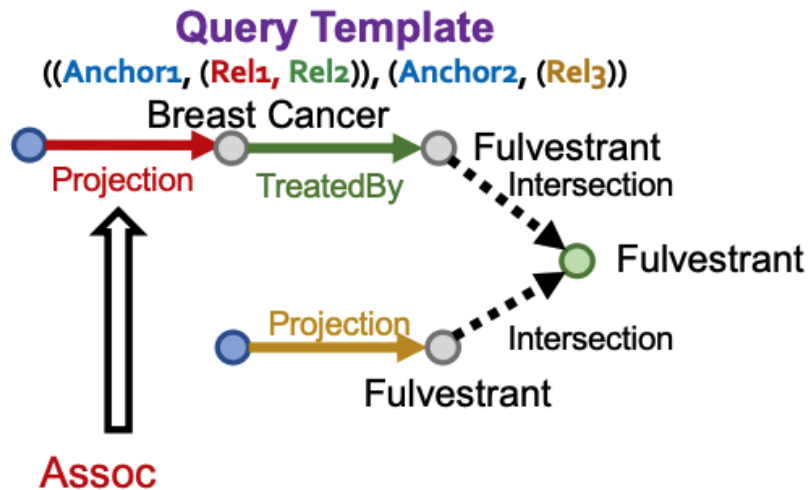
Pick one of the branches (top). Instantiate the **projection edge** in the template by randomly sampling one relation associated with the current entity, Fulvestrant. For example, we might select **TreatedBy** and then check what entities are connected to Fulvestrant with **TreatedBy**: {Breast Cancer}



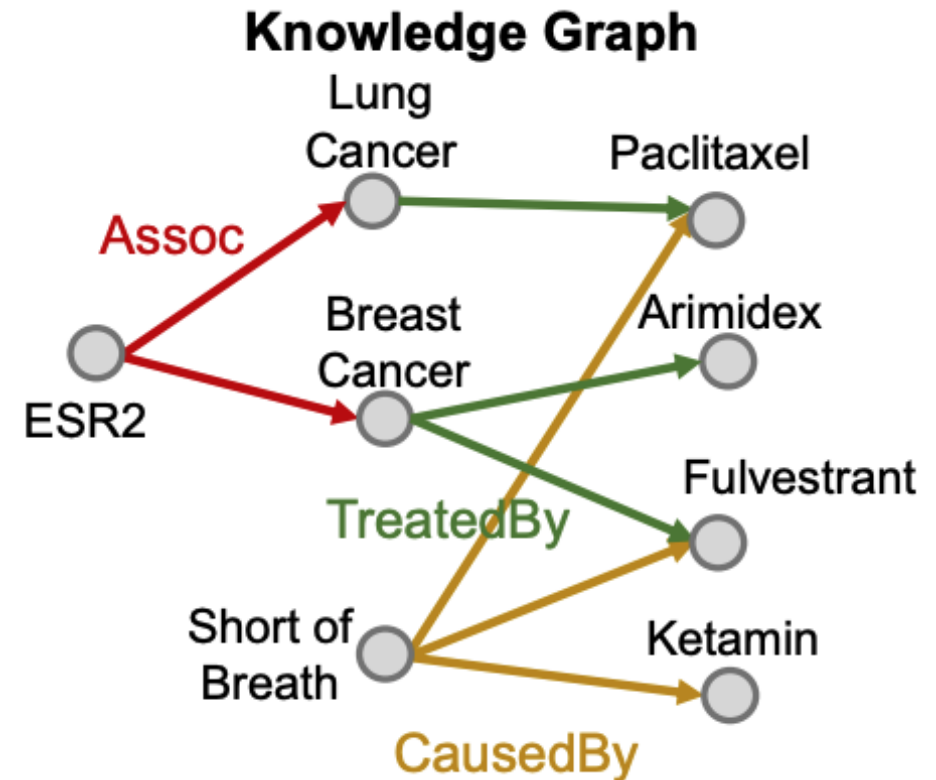


Generating sample queries on a KG

- How to do this systematically?



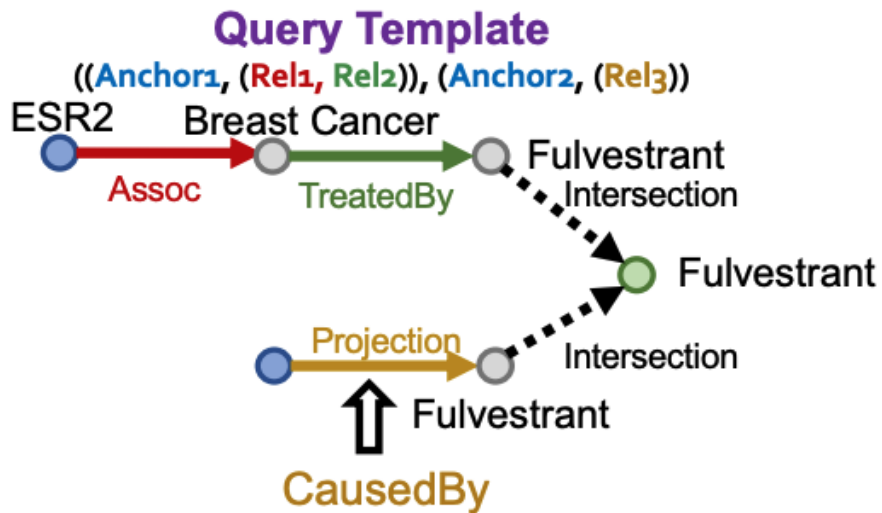
Staying on the top branch, instantiate the next **projection edge** by randomly sampling one relation associated with Breast Cancer. For example, we might select **Assoc** and then check what entities are connected to Breast Cancer with **Assoc**: {ESR2}



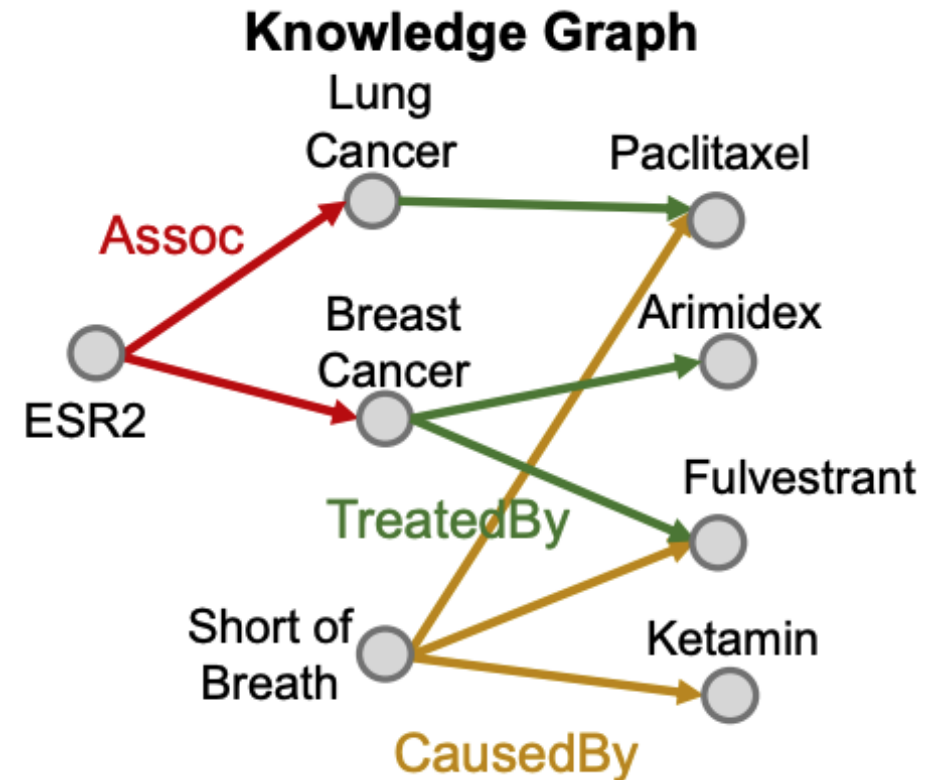


Generating sample queries on a KG

- How to do this systematically?



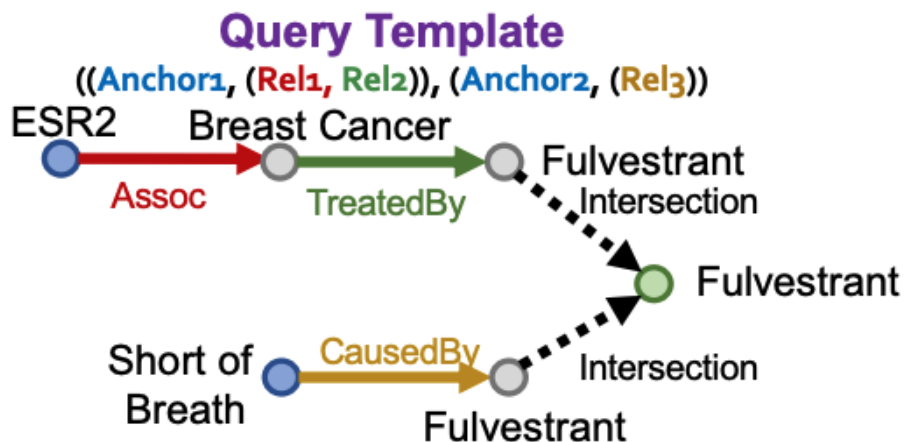
Now we move to the next (bottom) branch. We instantiate the **projection edge** by randomly sampling one relation associated with Fulvestrant. For example, we might select **CausedBy** and then check what entities are connected to Fulvestrant with **CausedBy**: {Short of Breath}



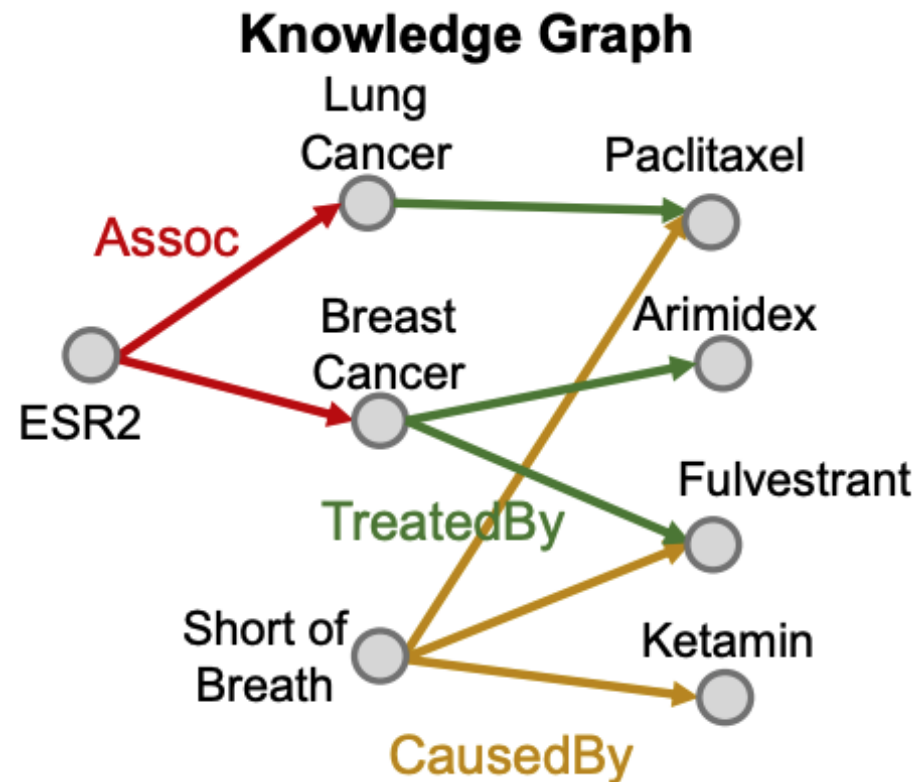


Generating sample queries on a KG

- How to do this systematically?



Finally, we select the last entity from {Short of Breath}





Generating sample queries on a KG

- We now have a fully instantiated query
 $[(e:\text{ESR2}, (r:\text{Assoc}, r:\text{TreatedBy})), (e:\text{Short of Breath}, (r:\text{CausedBy}))]$
- We can obtain the “known” answers by traversing the KG – this will NOT account for incompleteness
- We can then randomly select negative answers that are not in the known answers (there is the chance of false negatives)

